

Basi di Dati

- 01 Introduzione alle Basi di Dati
- 02 Progettazione Concettuale
- 03 Progettazione Logica
- 04 Normalizzazione
- 05 Introduzione al Linguaggio SQL

01

Introduzione alle Basi di Dati

Sistema Informatico vs Sistema Informativo

Informatica:

Scienza del **trattamento razionale**, specialmente per mezzo di **macchine automatiche**, **dell'informazione**, considerata come supporto alla conoscenza umana e alla comunicazione.

È quindi composta da due “anime”:

Tecnologica:
automatizzazione dei processi di calcolo

Metodologica:
**rappresentazione eleganti ed efficienti
della realtà**

Sistema Informativo

È l'insieme di tutte le **componenti** necessarie a **gestire** le **informazioni** di un'organizzazione:

- risorse **umane**
- **materiali**
- **procedimenti** manuali
- **regolamenti** e **norme** organizzative
- sistemi **informatici**
 - **componente tecnologica** principale di un sistema informativo!

Tuttavia, il concetto di sistema **informativo non dipende** da eventuali **automatizzazioni**:

- un S.I. può esistere anche senza sistemi informatici!

Sistema Informatico

È la **componente automatizzabile** dei Sistemi Informativi.

Informazione è quindi:

- trattata **automaticamente**
- rappresentata **digitalmente**

Obiettivo:

- **agevolare il funzionamento** del Sistema Informativo



Sistemi Operazionali e Business Intelligence

I sistemi informativi hanno due principali modalità di utilizzo:

- Sistemi **Operazionali**
 - **rivolti agli operatori**
 - **tempi** di risposta **brevi**
 - **organizzazione, utilizzo e gestione** “puntuale” dei **dati**
- Sistemi di **Business Intelligence**
 - **rivolta ai manager**
 - **tempi** di risposta **alti**
 - estrarre **informazioni utili** per prendere **decisioni**
 - **Profiling**
 - **Decision Support Systems**
 - Datawarehouse
 - Machine **Learning**/Data **Mining**

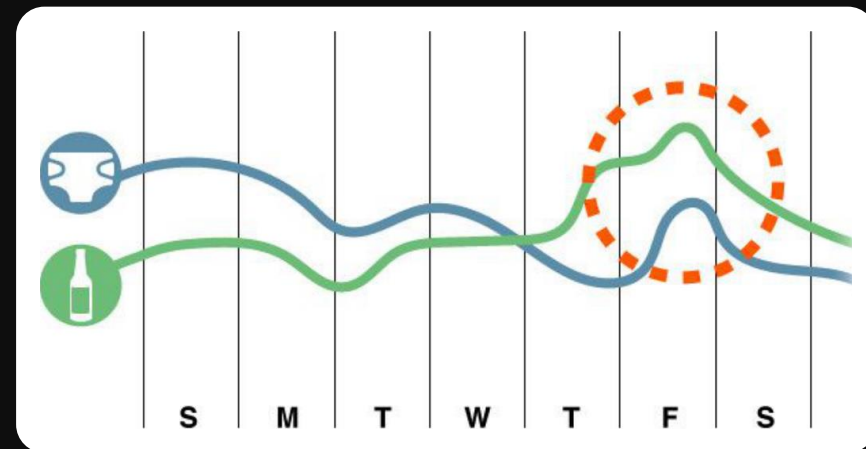


Business Intelligence

Dato = Risorsa **Strategica**

L'informazione è il più grande strumento per **esercitare potere** e **controllo** su qualsiasi **organizzazione**!

Dai dati si può estrarre **conoscenza aggiuntiva** per prendere **decisioni**!



Archivi

Archivio:

- **insieme di informazioni** su **oggetti** dello **stesso tipo** (dati omogenei)
 - composto da “**record**”
 - composti da “**campi**”
- informazioni **memorizzate** su memoria **persistente***
- utilizza **criteri** per garantire
 - **assenza di errori**
 - **reperimento rapido** delle informazioni



* Archivio != File

- Documento != Foglio di carta
- un archivio **astrae** dai file!

Da Archivio a Database

Un **archivio** è strettamente legato al **software** che lo **utilizza**:

- spesso è **implementato ad-hoc** per l'applicazione
- soltanto il **programmatore** sa
 - come è **organizzato**
 - **non** segue uno **schema standard**
 - come **prendere i dati**
 - **non** utilizza un “**linguaggio standard**” di **interrogazione**

Un DataBase Management System (DBMS) si mette **tra** il **software** e il supporto di **archiviazione** (locale o remoto):

- consente di **svincolarsi** dal **programmatore**
- consente ai **programmatori** di **svincolarsi** dai **file**
- segue **schemi** di rappresentazione e **linguaggi** di interrogazione **standard**

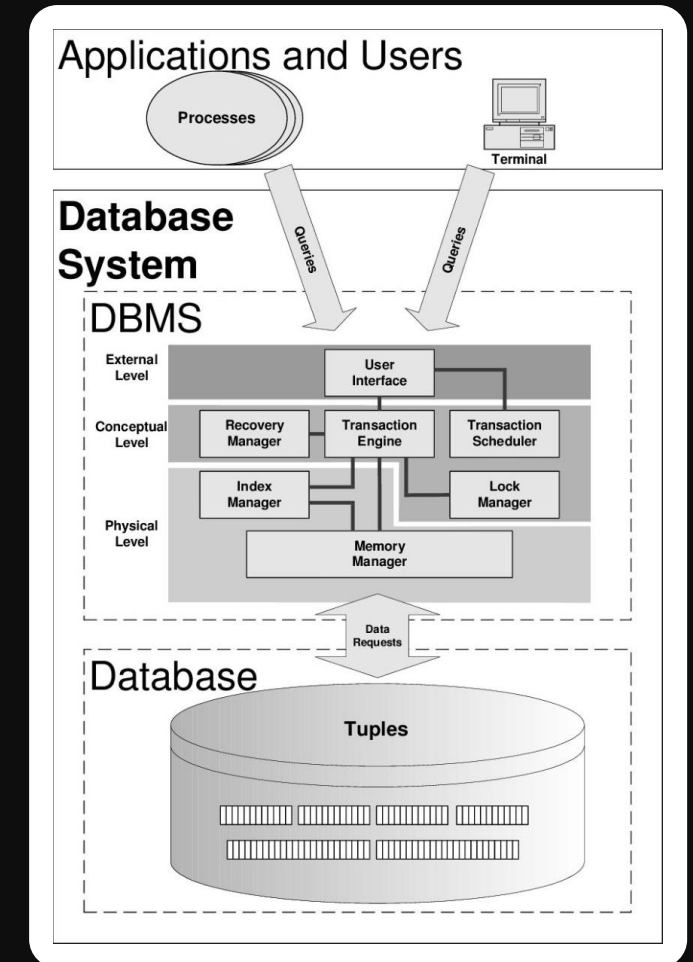
Database e DBMS

Un **Database** è una **raccolta** di dati:

- **omogenei**
- logicamente **correlati**
- che **modellano** una **realtà** di interesse
- **conservati** su **supporto** di **archiviazione**

Un DBMS è quindi un sistema **complesso** capace di **gestire** collezioni di **dati** (database):

- **grandi**
- **persistenti**
- **condivise**



DBMS

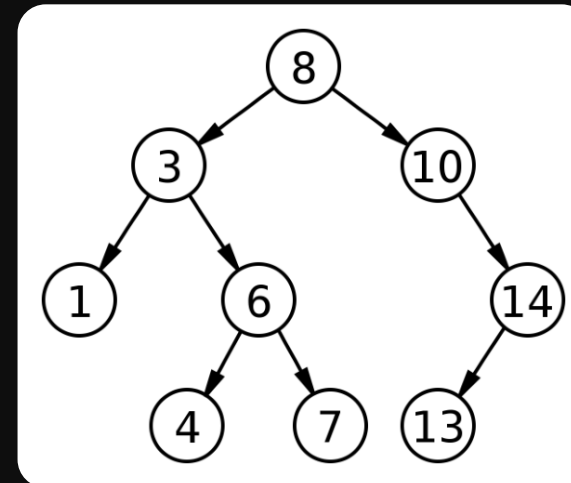
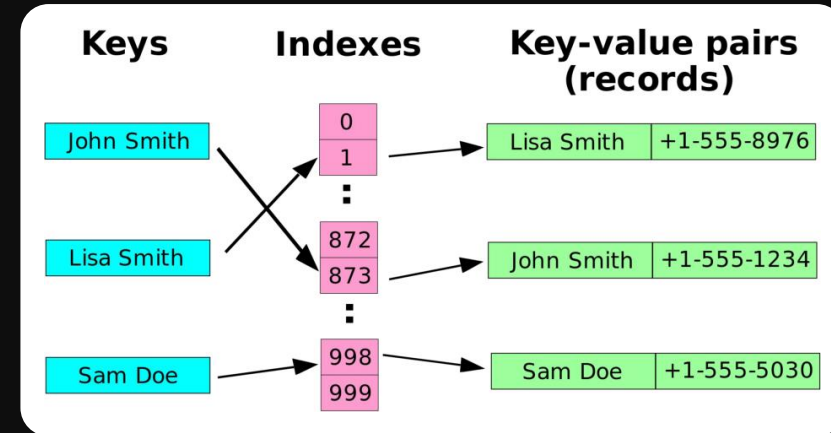
Un DBMS deve gestire le operazioni sui dati in modo **efficace**:

- **accesso**
 - **concorrente** tra più applicazioni
 - meccanismi per **permessi** e **priorità**
- **affidabilità**
 - garantire l'**integrità** dei dati
 - meccanismi di **ridondanza** e/o **backup**
 - **consistenza** (coerenza) dei dati tra diverse sezioni del database
- **efficienza**
 - **tempo** di risposta **basso**
 - **scalabilità**
 - le **dimensioni** dei DB possono essere di gran lunga **maggiori** di quelle della **RAM**!

Organizzazione del DBMS

Per ottenere tali proprietà, un DBMS supporta diverse modalità di **gestione** dei **dati** con:

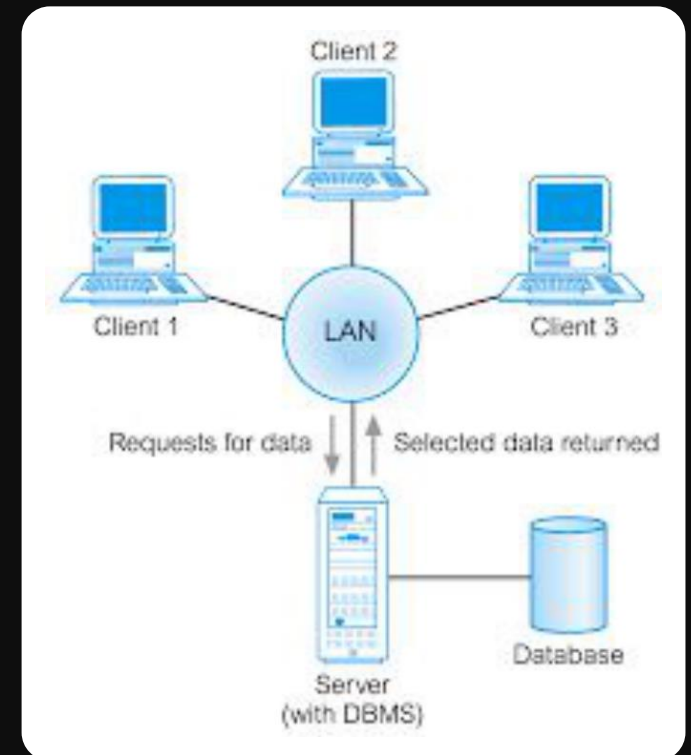
- strutture **sequenziali**
 - file di **testo**
 - file **binari** di “record”
- strutture **indicizzate** (accesso veloce tramite “chiave”)
 - **hash** map
 - **alberi**



Architettura dei Database

Disaccoppiando la **logica** di gestione dei dati dalle applicazioni, per un **software** non sarà importante **dove** si trova il **DBMS** (a patto che sia accessibile)

- DBMS sul **client**
 - tempi di **risposta bassi**
 - più **economico** (a breve termine)
 - scarsa **scalabilità**
- DBMS su **server** dedicato (rete locale o remoto)
 - stessi **dati accessibili** facilmente da **più applicazioni**
 - tengo tutte le **informazioni** nello **stesso “posto”**
 - **dipendenza** dalla **rete** (internet o lan)
 - possibili **problemi** di **sicurezza**
 - maggior **latenza**



Progettazione basi di dati

Analisi della realtà di interesse:

- **Specifiche** sulle funzioni (ling. naturale)
- **Analisi** delle funzioni
- **Specifiche formali** delle funzioni
- **Progetto** delle funzioni
- **Pseudocodice**

Focus sulla **manipolazione** dei **dati**

Progettazione DB!

- **Specifiche** sui dati (ling. naturale)
- **Analisi** dei dati
- **Specifiche formali** dei **dati**
- **Progetto** dei **dati**
- **Tabelle**

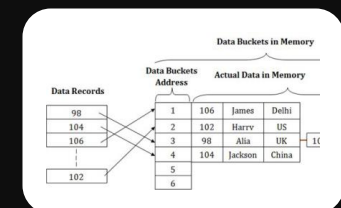
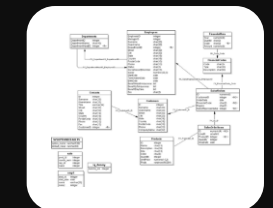
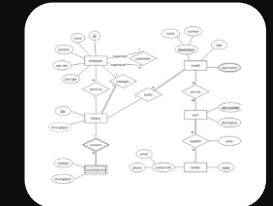
Focus sulla **rappresentazione** dei **dati**



Progettazione basi di dati

Obiettivo: **rappresentazione** della realtà per gestire le informazioni in modo **efficiente** ed **efficace**

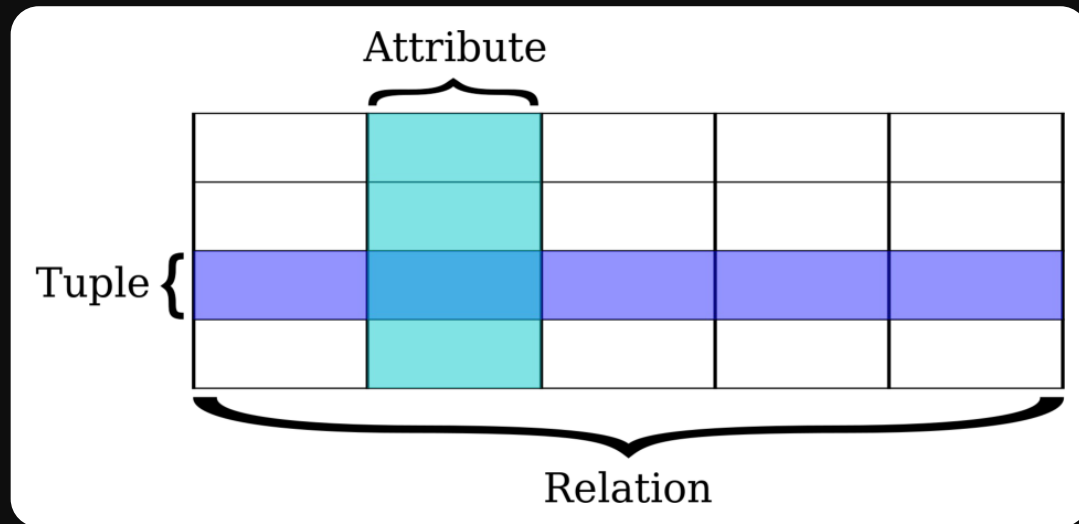
- Progettazione **concettuale**
 - schema o modello concettuale
 - formalismi **grafici** per esprimere **oggetti**, **proprietà** e **legami tra dati**
 - **schemi ER** o diagrammi UML
- Progettazione **logica**
 - schema o modello logico
 - rappresentazione dei dati mediante **tabelle**
 - **implementabili** con un **DBMS**
- Progettazione **fisica**
 - schema o modello fisico
 - **implementazione** del modello logico tenendo conto delle **prestazioni effettive** e del **DBMS in uso**



Database Relazionali

Nei Database “classici”, per rappresentare i dati si utilizza il concetto di “**Relazione**” (in inglese Relation).

Prendono infatti il nome di RDBMS (Relational DBMS).



Relazione = Insieme di Tuple/Record

Tupla / Record = $\langle d_1, d_2, \dots, d_n \rangle$

Ogni **elemento** d_i **appartiene** ad un **dominio** D_i (tipo)

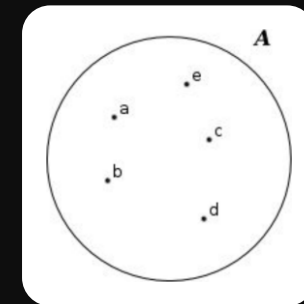
Relazioni, tuple ed insiemi

A livello **pratico** una **relazione** può essere pensata come una “**tabella**” composta da una serie di **righe (tuple)** con un insieme **colonne (attributi)**.

Dominio / Tipo		Attributo / Colonna
ID (Integer)	Name (String)	Address (String)
102	Yonezawa Akinori	Naha, Okinawa
202	Nilay Patel	Sendai, Miyagi
104	Murata Makoto	Kumamoto, Kumamoto
152	Matsumoto Yukihiro	Okinawa, Okinawa

Tupla / Record/ Riga

Relazione / Tabella



Relazione = Insieme.

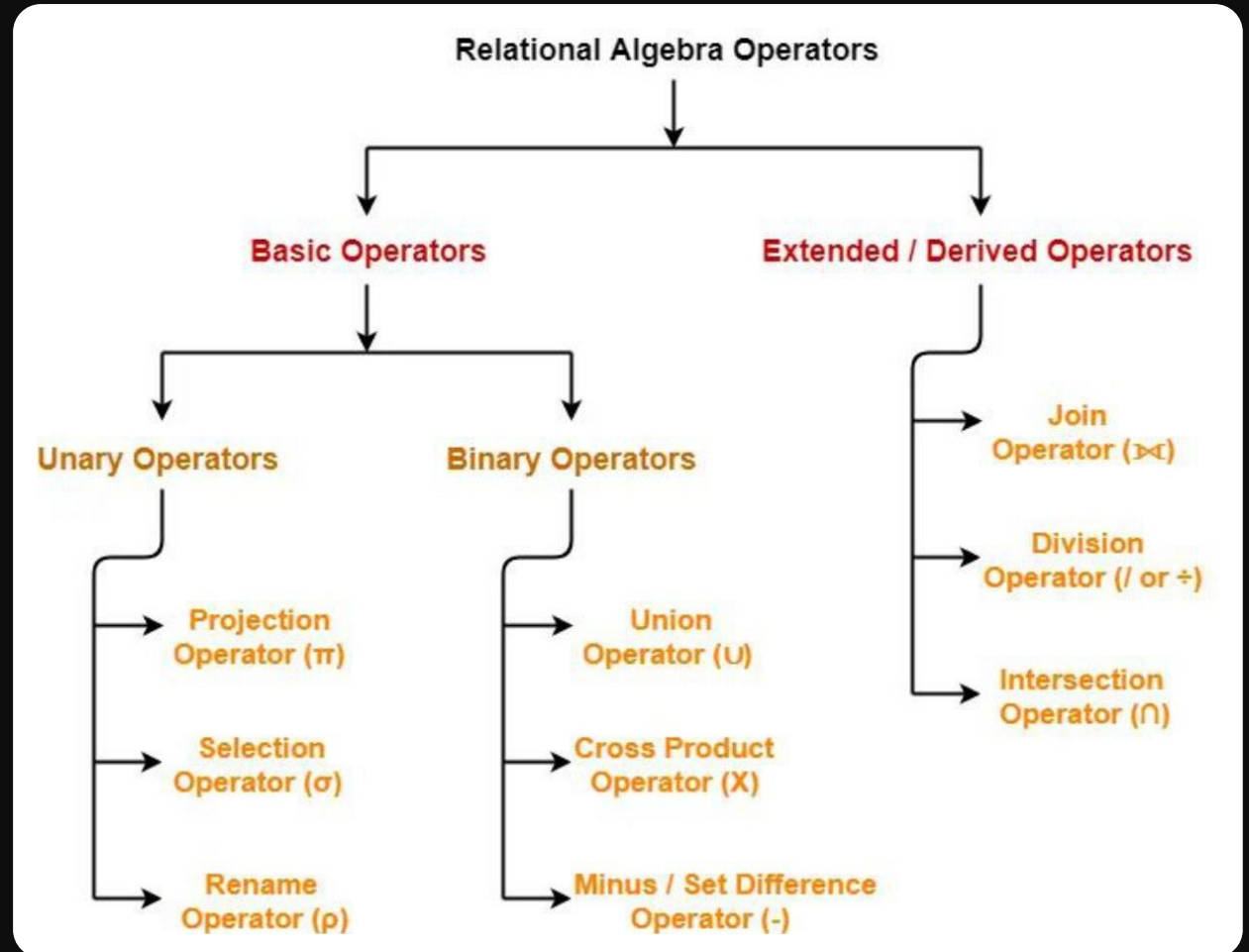
Ogni **tupla** è un **elemento** dell'insieme

Algebra Relazionale e Operazioni

Un DB relazionale è quindi **composto** da una serie di **relazioni** che **modellano** una certa **realtà** di interesse.

Le **relazioni** possono essere **trasformate** e **combinare** per ottenere nuove **informazioni**.

I **meccanismi** per la **manipolazione** delle relazioni sono descritti **dall'Algebra Relazionale**.



Algebra Relazionale e Operazioni

Le operazioni si **classificano** in:

- **primitive** / di base
 - non possono essere espresse in funzione di altre operazioni
- **derivate**
 - sono definite come combinazione di altre operazioni

Si distinguono anche in base al **numero** di **operandi** richiesti:

- **unarie**
 - lavorano su una sola relazione
- **binarie**
 - lavorano su due relazioni

Il **risultato** dell'applicazione di uno degli operatori è una **nuova relazione**!

Operazioni Primitive Unarie (Algebra Relazionale)

Proiezione (Π - pi)

- presa una relazione R ,
- ed un insieme di attributi $\{a_1, \dots, a_n\}$,
- **restringe** le tuple di R **all'insieme di attributi** specificato

$$\Pi_{a_1, \dots, a_n}(R)$$

Intuitivamente:

- “**sezione verticale**” della tabella

Person			$\Pi_{\text{Age, Weight}}(\text{Person})$	
Name	Age	Weight	Age	Weight
Harry	34	180	34	180
Sally	28	164	28	164
George	28	170	28	170
Helena	54	154	54	154
Peter	34	180		

Keyword **SQL**: SELECT

Operazioni Primitive Unarie (Algebra Relazionale)

Selezione (σ - sigma)

- presa una relazione R ,
- ed un'espressione logica ϕ (phi)
- calcola il **sottoinsieme** di **tuple** di R per cui l'espressione ϕ è vera

$$\sigma_{\phi}(R) = \{ t : t \in R, \phi(t) \}$$

Intuitivamente:

- **condizione** sulle tuple
- “**sezione orizzontale**” della tabella

Person

Name	Age	Weight
Harry	34	80
Sally	28	64
George	29	70
Helena	54	54
Peter	34	80

$\sigma_{\text{Age} \geq 30 \wedge \text{Weight} \leq 60}(\text{Person})$

Name	Age	Weight
Helena	54	54

Keyword **SQL**: WHERE

Operazioni Primitive Unarie (Algebra Relazionale)

Rinominazione (ρ - rho)

- presa una relazione R ,
- ed un attributo “b”
- ed un nuovo nome per l’attributo “a”
- **rinomina** l’attributo “b” in “a” in tutte le tuple

$$\rho_{a/b}(R)$$

Intuitivamente:

- **cambia** il **nome** di una **colonna**

Employee		$\rho_{\text{EmployeeName/Name}}(\text{Employee})$	
Name	EmployeeId	EmployeeName	EmployeeId
Harry	3415	Harry	3415
Sally	2241	Sally	2241

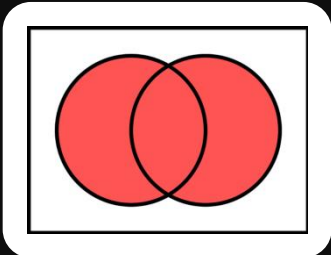
Keyword **SQL**: AS

Operazioni Primitive Binarie (Algebra Relazionale)

Unione (U):

- date due **relazioni** A e B
- implementa l'**unione insiemistica**
- A e B devono essere “**compatibili**”
 - stessi attributi

$$A \cup B = \{x : x \in A \text{ or } x \in B\}$$



$$|A \cup B| = |A| + |B| - |A \cap B|$$

|A| indica la **cardinalità** dell'insieme, cioè il # di elementi

Course_1

C_id	C_name
11	Foundation C
21	C++
31	JAVA

Course_2

C_id	C_name
12	Python
21	C++

Course_1 U Course_2

C_id	C_name
11	Foundation C
21	C++
31	JAVA
12	Python

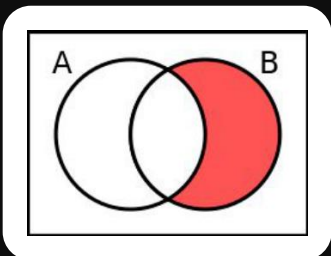
Keyword **SQL**: UNION

Operazioni Primitive Binarie (Algebra Relazionale)

Differenza (\ oppure -):

- date due **relazioni** A e B
- implementa la **differenza (complemento) insiemistica**
- A e B devono essere “**compatibili**”
 - stessi attributi

$$B \setminus A = \{x \in B : x \notin A\}.$$



$|B \setminus A| = |B| - |A \cap B|$
 $|A|$ indica la **cardinalità**
 dell'insieme, cioè il # di elementi

Course_1

C_id	C_name
11	Foundation C
21	C++
31	JAVA

Course_2

C_id	C_name
12	Python
21	C++

Course_1 - Course_2

C_id	C_name
11	Foundation C
31	JAVA

Keyword **SQL**:

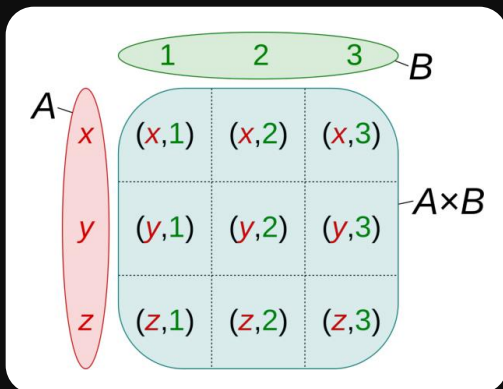
- EXCEPT
- NOT IN

Operazioni Primitive Binarie (Algebra Relazionale)

Prodotto cartesiano ($\times$):

- date due **relazioni** A e B
- implementa il **prodotto insiemistico**
- crea nuove tuple **combinando** gli **attributi** di A e B

$$A \times B = \{(a, b) \mid a \in A \text{ and } b \in B\}.$$



Intuitivamente:

- **combina** tutti gli elementi di A con tutti quelli di B

$$|A \times B| = |A| * |B|$$

Student

S_id	Name	Class	Age
1	Andrew	5	25
2	Angel	10	30
3	Anamika	8	35

Course

C_id	C_name
11	Foundation C
21	C++

Student X Course

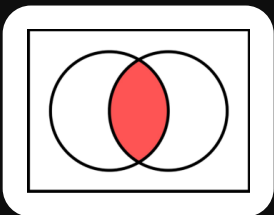
S_id	Name	Class	Age	C_id	C_name
1	Andrew	5	25	11	Foundation C
1	Andrew	5	25	21	C++
2	Angel	10	30	11	Foundation C
2	Angel	10	30	21	C++
3	Anamika	8	35	11	Foundation C
3	Anamika	8	35	21	C++

Keyword **SQL**: FROM <RelA>,<RelB>

Operazioni Derivate Binarie (Algebra Relazionale)

Intersezione ($\cap$):

- date due **relazioni** A e B
- implementa l'**intersezione insiemistica**
- A e B devono essere “**compatibili**”
 - stessi attributi
- **non** primitiva
 - può essere espressa in funzione di altri operatori
 - $A \cap B = (A \cup B) \setminus (A \setminus B) \setminus (B \setminus A)$



Course_1

C_id	C_name
11	Foundation C
21	C++
31	JAVA

Course_2

C_id	C_name
12	Python
21	C++

Course_1 $\cap$ Course_2

C_id	C_name
21	C++

Keyword SQL:

- INTERSECT
- IN

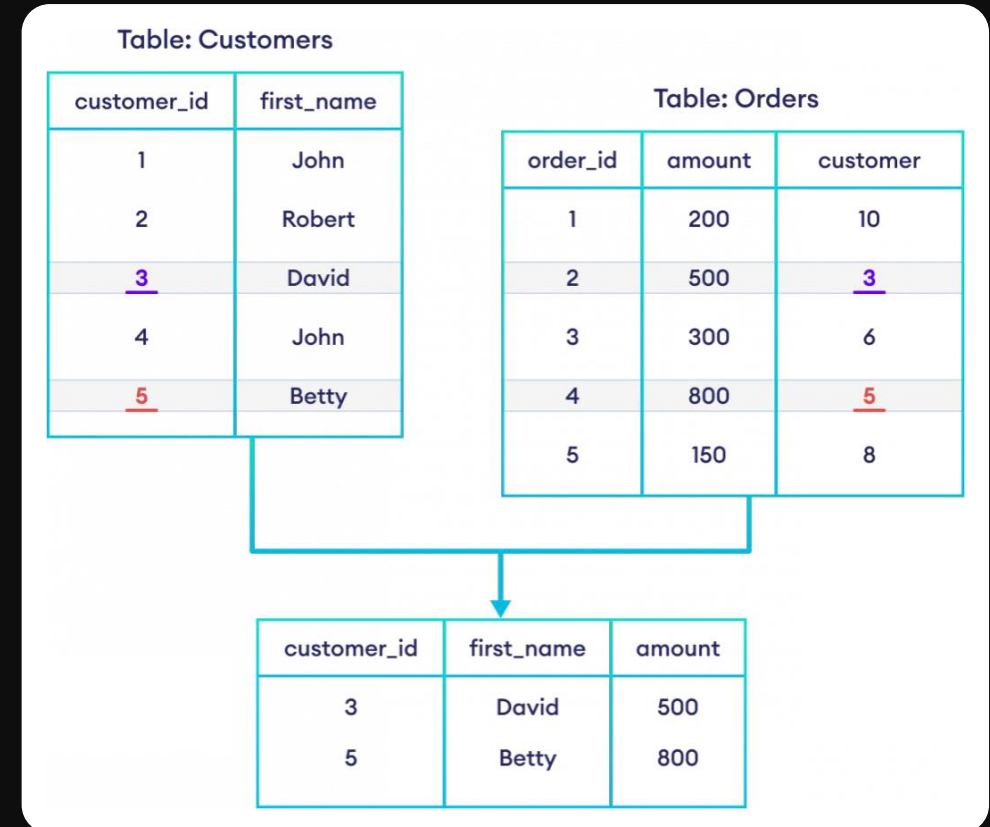
Operazioni Derivate Binarie (Algebra Relazionale)

Join ($\bowtie$):

- date due **relazioni** A e B
- ed un **attributo** “x” in comune con entrambe
- ne implementa la “**giunzione**” sull’attributo “x”
- **non** primitiva
 - può essere espressa in funzione di altri operatori
 - prodotto cartesiano seguito da proiezione e selezione

“**Combina**” in maniera più **efficiente** del prodotto cartesiano!

Non genera righe e colonne inutili.



Keyword **SQL**: JOIN

02

Progettazione Concettuale

Progettazione Concettuale

Processo volto alla **realizzazione** del **modello** concettuale, sfruttando diverse **fonti** del **sistema informativo** in esame:

- interviste
- documentazione
 - prodotta appositamente
 - già esistente (normative, regolamenti, processi aziendali, ecc...)
 - modulistica

Queste informazioni vengono tipicamente **raccolte** in una **fase iniziale** detta “**analisi dei requisiti**”.

Modello Concettuale

È una **rappresentazione astratta** e **formale** del dominio di applicazione.

- punto di incontro tra il **progettista** del DB e il “**cliente**”
- **indipendente** dal sistema di **memorizzazione**
 - si vuole **ragionare** sui **dati** e sulle loro **correlazioni**

Si utilizzano **formalismi grafici** standard come:

- **Entity Relationship diagram** (diagramma entità associazioni)
- Unified Modeling Language (UML)

Entità e Associazioni

Si compone di due blocchi fondamentali:

- **entità**
 - **classe** di oggetti (concreti o astratti) che ha **senso** anche se “**isolata**” dal resto
 - **insieme** di **oggetti simili** (dati dello stesso tipo, o con caratteristiche simili, raccolti insieme)
 - è di **interesse** per la **realtà** da modellare
 - termini (**nomi**/sostantivi) scelti al **singolare**!
- **associazioni** (relationships)
 - **legame** tra **entità**
 - da **non confondere** con “**relation**” (relazione)
 - termini (**verbi**) scelti **all’infinito** o al **presente**

Ad esempio, nel S.I. scuola, volendo **rappresentare** i **prof**, e le relative **materie insegnate**, si ha:

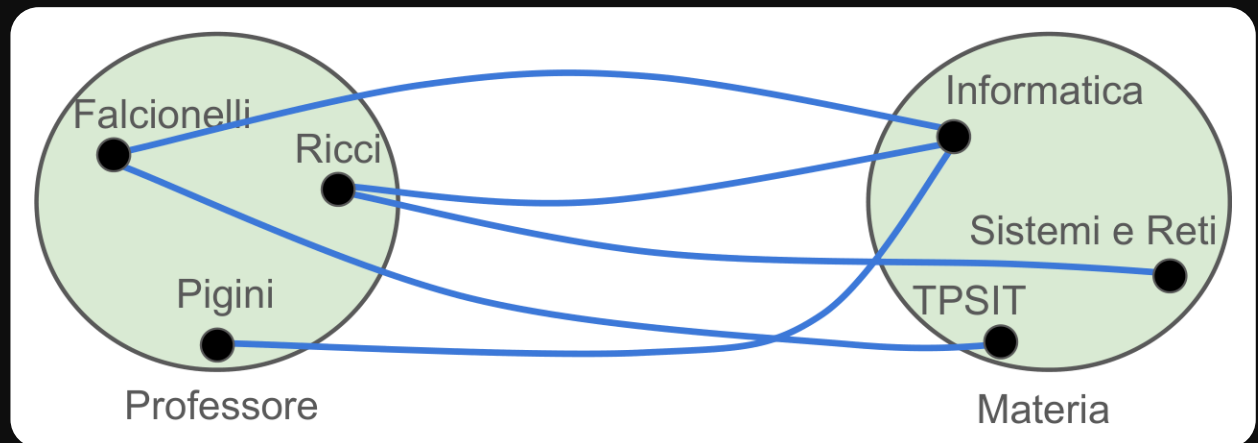
- **entità**:
 - professore
 - materia
- **associazioni**:
 - insegna/insegnare

Entità e Associazioni

- **entità**
 - sono composte da **attributi**
 - ogni riga è **un'istanza** dell'entità
 - sinonimi: tabella, classe, **relazione**(!!)
- **associazioni** (relationships)
 - **collegano** le **istanze** di un'entità a quelle di **un'altra** entità



Se entità = relazione ...
... entità = insieme di tuple!



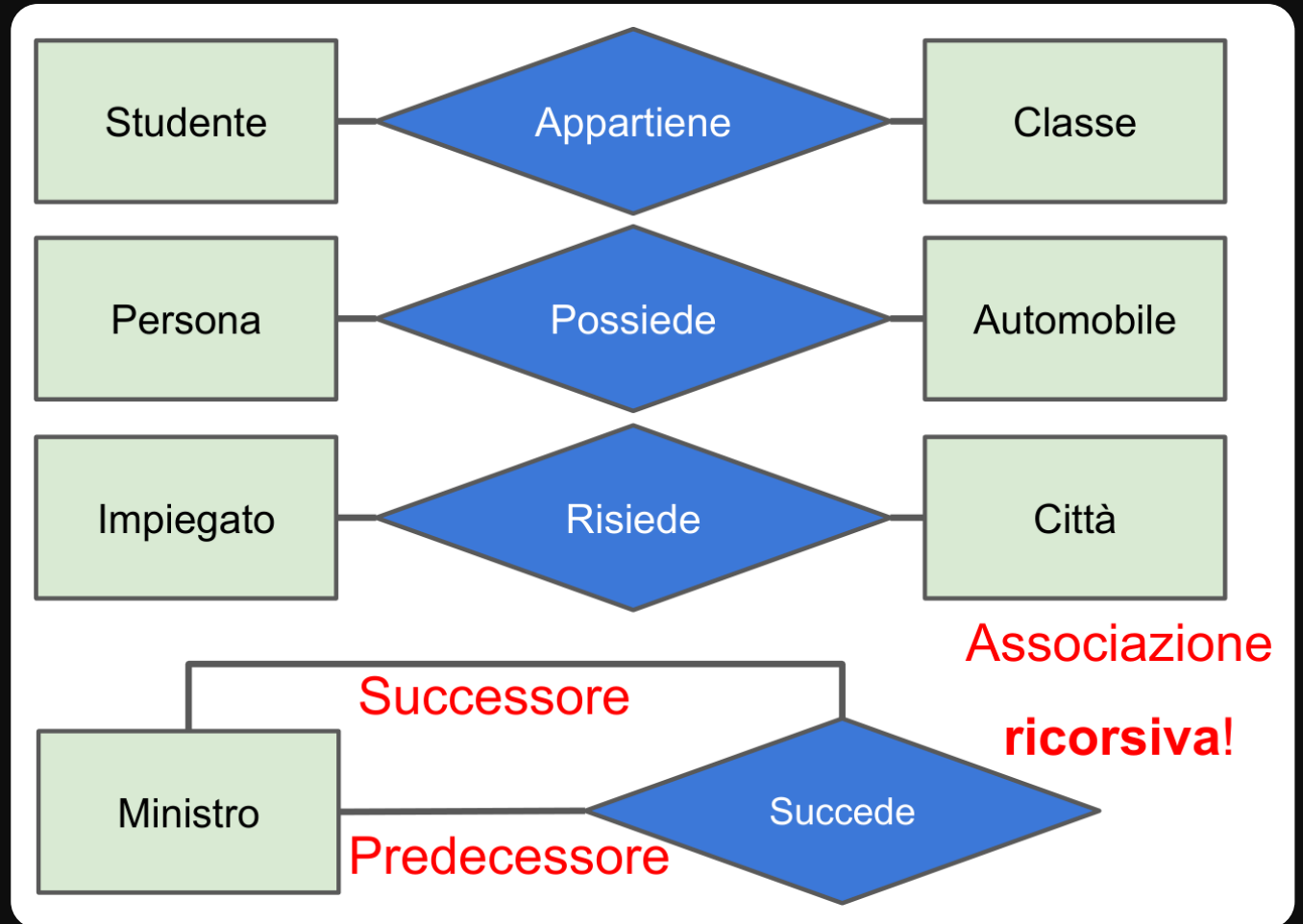
Esempi di Entità e Associazioni

Ogni studente appartiene alla rispettiva classe...

Ogni persona può possedere delle automobili...

Ogni impiegato ha la propria città di residenza...

Ogni ministro è susseguito da un nuovo ministro...

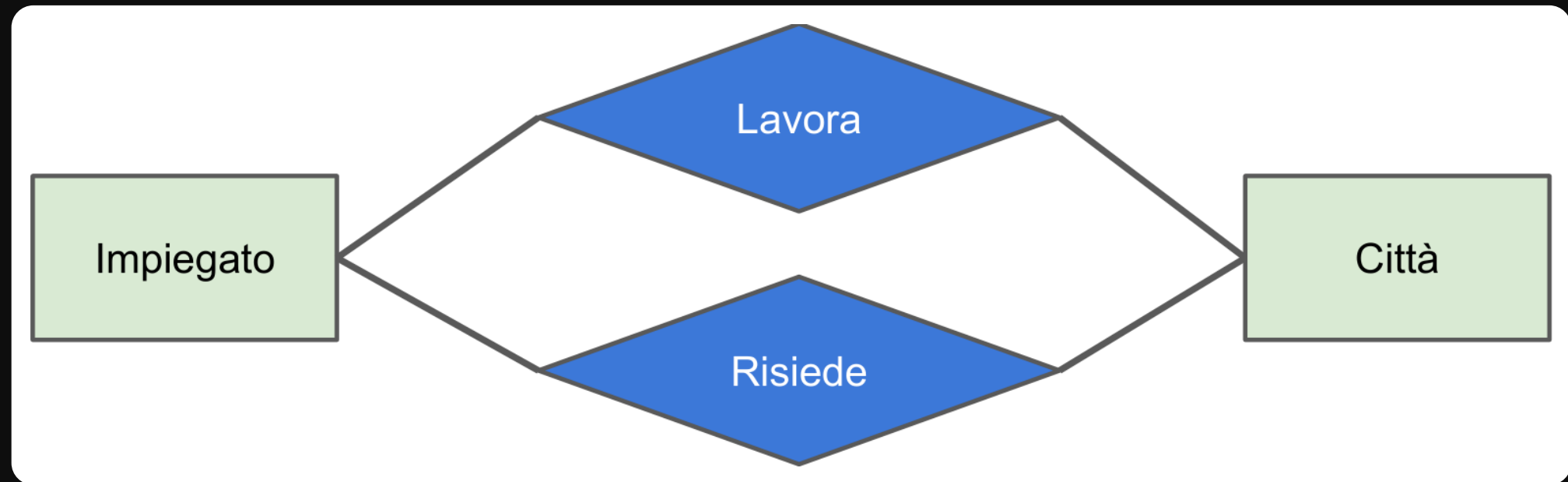


Entità e Associazioni Multiple

Tra due entità possono esserci **più associazioni**.

Ad esempio:

- Ogni impiegato ha una città di residenza ed una città dove lavora.



Cardinalità

Come si possono “**quantificare**” le associazioni?

Date due relazioni A e B, ci si chiede:

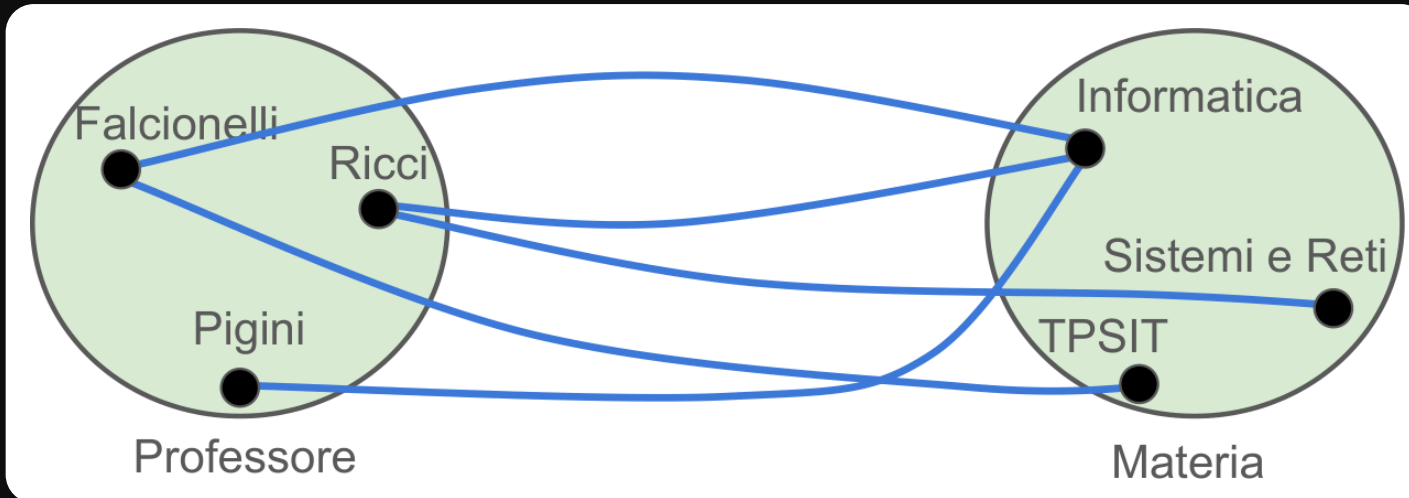
- data **un’istanza** di **A**, **quante istanze** di **B** sono associate ad essa?
- data **un’istanza** di **B**, **quante istanze** di **A** sono associate ad essa?

Si considerano quindi:

- il **minimo** ed il **massimo** numero di collegamenti **da** un’istanza di **A** ad una di **B**
 - cardinalità minima e massima da A a B
- il **minimo** ed il **massimo** numero di collegamenti **da** un’istanza di **B** ad una di **A**
 - cardinalità minima e massima da B ad A

Ad **ogni associazione** corrispondono **due coppie** di valori!

Cardinalità



cardinalità da **Professore** a **Materia**:

- minima: 1
- massima: 2

cardinalità da **Materia** a **Professore**:

- minima: 1
- massima: 3



Cardinalità

Per ovvie ragioni, sarebbe **proibitivo** “contare” **manualmente** tutte le **occorrenze** di **ogni associazione** dello schema concettuale.

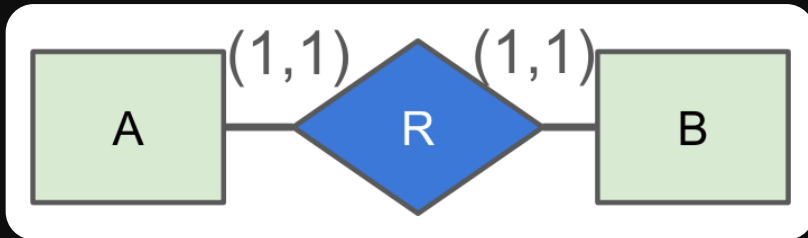
Quindi, a meno che questi numeri non siano esplicitati da qualche parte nel S.I. in esame, le **cardinalità** si **indicano** in maniera “**generica**”

- cardinalità **minima**
 - 0: corrispondenza **opzionale**
 - 1: corrispondenza **obbligatoria**
- cardinalità **massima**
 - 1: corrispondenza **bi-univoca**
 - N: corrispondenza **multipla**



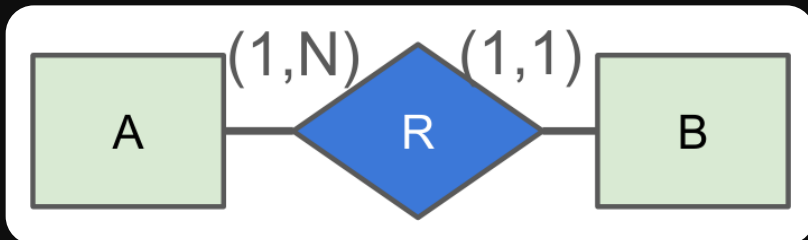
Quante **combinazioni possibili** esistono?

Cardinalità



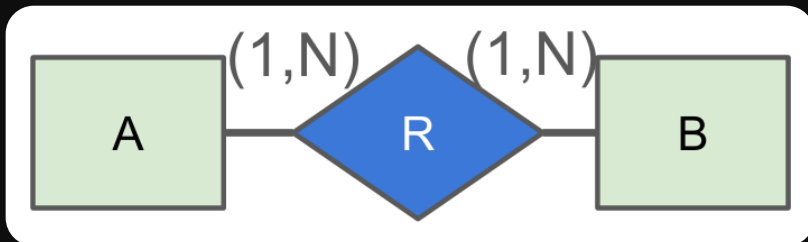
Associazione **bi-univoca (1-1)**:

ogni istanza della prima entità è associata ad una sola istanza della seconda, e viceversa



Associazione **semplice (1-N)**:

ogni istanza della prima entità è associata ad una o più istanze della seconda, mentre ogni istanza della seconda ad una sola della prima



Associazione **Complessa (N-N)**:

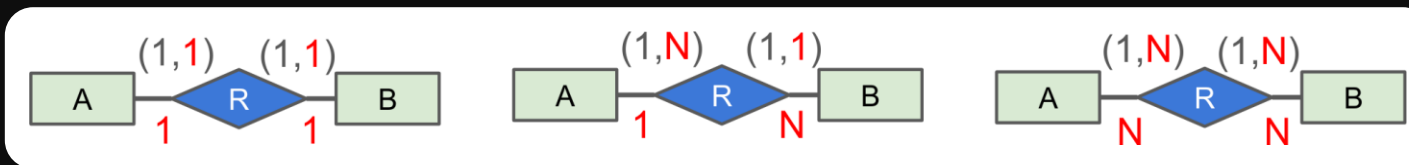
ogni istanza della prima entità è associata ad una o più istanze della seconda, e viceversa

Molteplicità

Nonostante la **cardinalità** derivi direttamente dall'interpretazione **insiemistica** delle **relazioni** e **associazioni**, talvolta sono considerate:

- poco “**compatte**”
- poco **intuitive** nella **lettura** dello schema concettuale

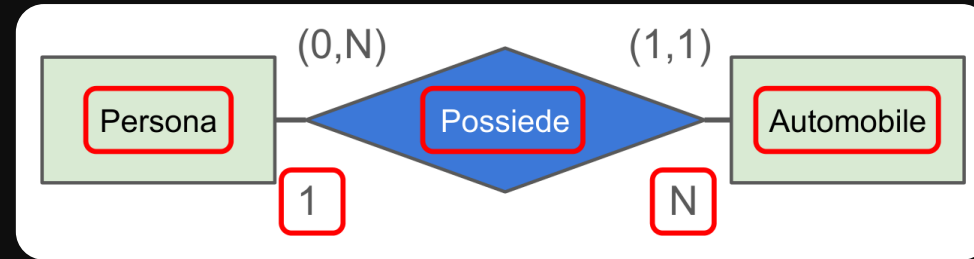
Per questi motivi, a partire dalla **cardinalità massima** si ricava la **molteplicità**:



Si indicano in maniera “**invertita**” rispetto alla cardinalità per **favorire** la **leggibilità**

Cardinalità vs Molteplicità

Con le molteplicità le associazioni si **leggono facilmente...**



- Da sx a dx: 1 Persona possiede N Automobili
- Da dx a sx: N Automobili sono possedute da N Persone

A parte una miglior compattezza e leggibilità, la **molteplicità non considera** le cardinalità **minime...**

L'importante è **sceglierne** una ed essere **coerenti**!

Noi utilizzeremo le **cardinalità** perché **meno ambigue** e vicine all'interpretazione **insiemistica**.

Attributi

Le **proprietà elementari** di **entità** ed **associazioni** vengono descritte dagli **attributi**:

- **formato** (o dominio)
 - **cardinalità** (# di valori che può assumere nel dominio)
 - da non confondere con quella delle associazioni!
 - esempio: il genere (M/F)
- **dimensione** (# di caratteri, # di cifre, ...)
- **opzionalità** (obbligatorio o no, “nullable”)

Sinonimi:

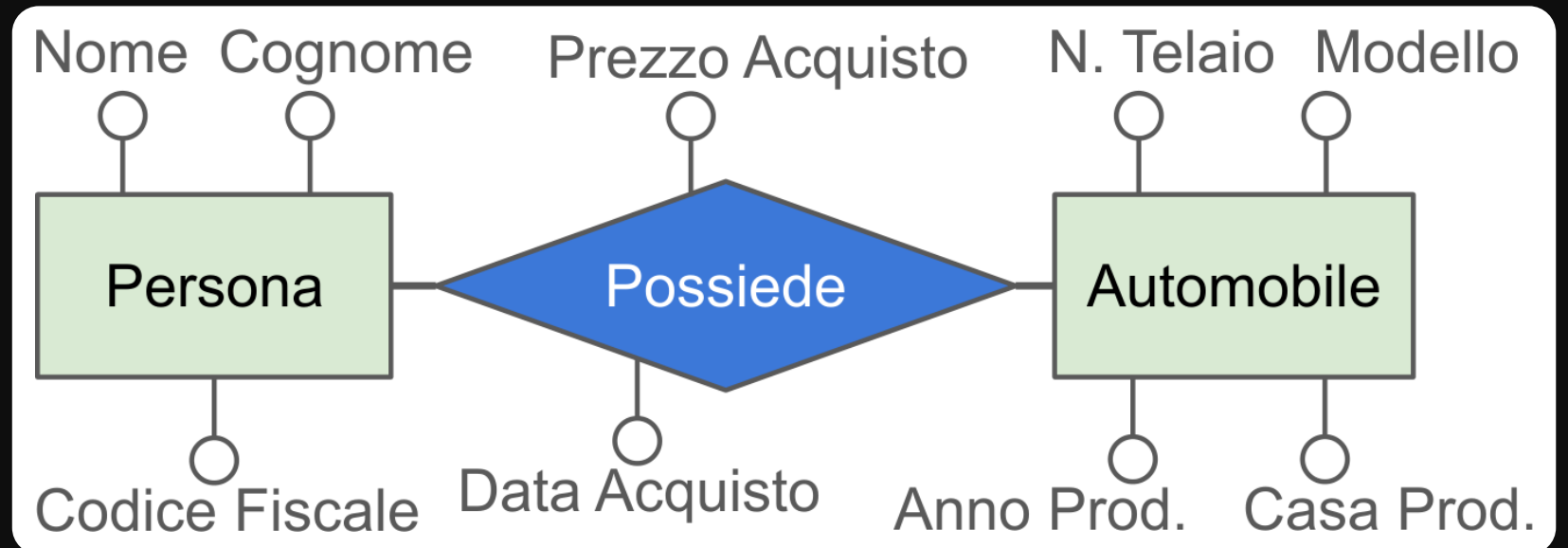
- campi
- proprietà
- caratteristiche

Considerando relazioni ed associazioni in forma **tabulare**, gli **attributi** diventano **colonne**!

Attributi

Graficamente, ogni attributo viene indicato con un **cerchio** collegato alla rispettiva entità o associazione. Il **colore** ne indica il **ruolo**:

- **bianco**
 - descrittivo
 - informativo
- **nero**
 - chiave
 - identificatore



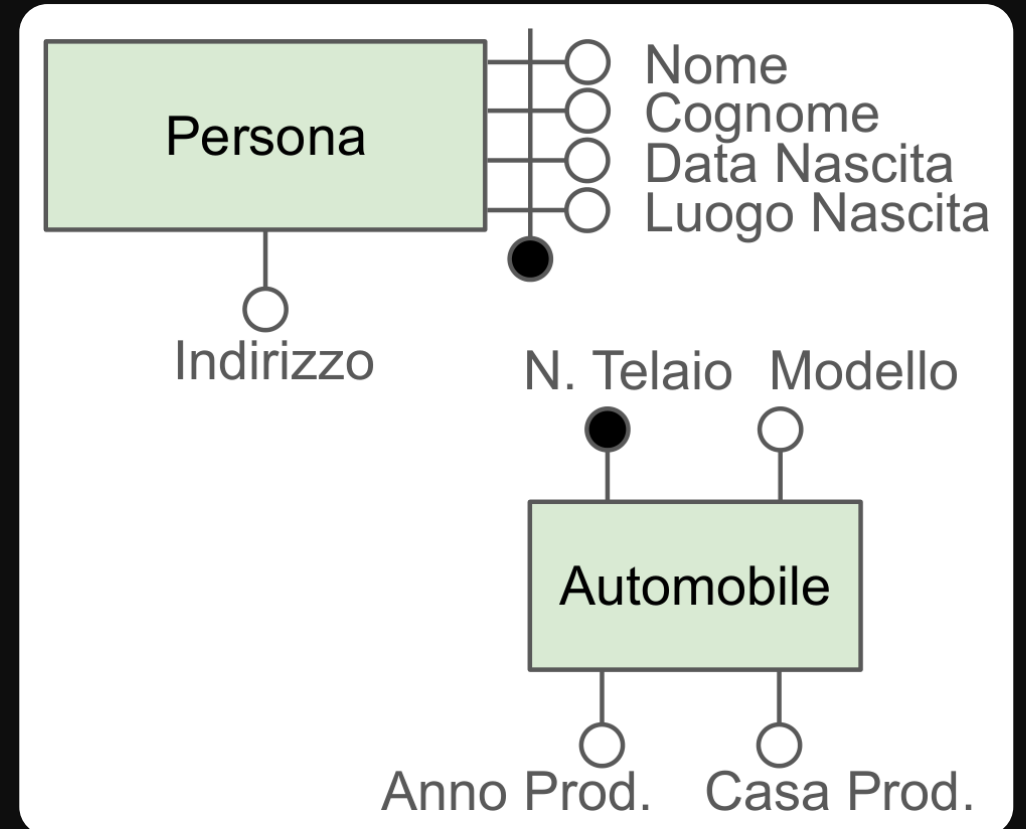
Attributo chiave o Identificatore

Ogni **tupla** deve poter essere **distinta univocamente** dalle altre.

Per questo motivo, occorre **individuare** un **insieme** di **attributi** che permette di **identificare** le istanze delle entità:

- **chiave primaria o identificatore**
- **insieme** minimale di **attributi**
 - tipicamente uno solo!
- **graficamente** si usa un **cerchio nero**:
 - **collegato** all'entità a cui si riferisce
 - **collegato** agli **attributi** che compongono la chiave

Nel modello ER, le associazioni non hanno attributi chiave!



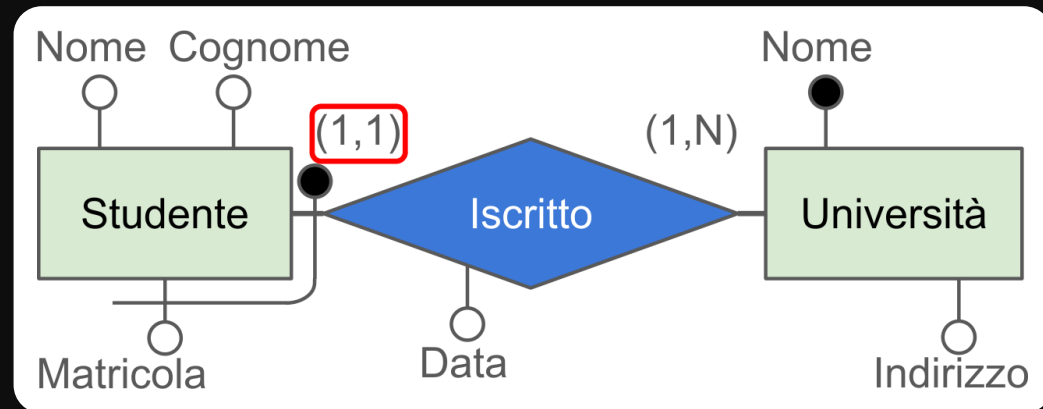
Identificatore interno ed esterno

Tuttavia, alcuni **attributi** hanno senso come **chiave** soltanto se **considerati** nel contesto di un'associazione.

Ad esempio, nel caso di **studenti iscritti all'università**, si avrebbe:

Che **problema** si avrebbe considerando soltanto “**matricola**” come **chiave**?

Tra università **diverse** si può avere **stessa matricola**!



Applicabile se c'è **cardinalità 1,1**

Entità deboli e Chiavi artificiali

Entità forti

- è **possibile** trovare una **chiave**

Entità deboli

- **non** è **possibile** trovare una **chiave**

In quest'ultimo caso, si devono **introdurre chiavi "artificiali"** ottenute inserendo un **attributo fittizio**:

- chiamato **"ID"**
- **contatore** progressivo incrementato ad ogni **nuova tupla**

A volte, si **inserisce** anche su entità **forti** per questioni di **comodità!**

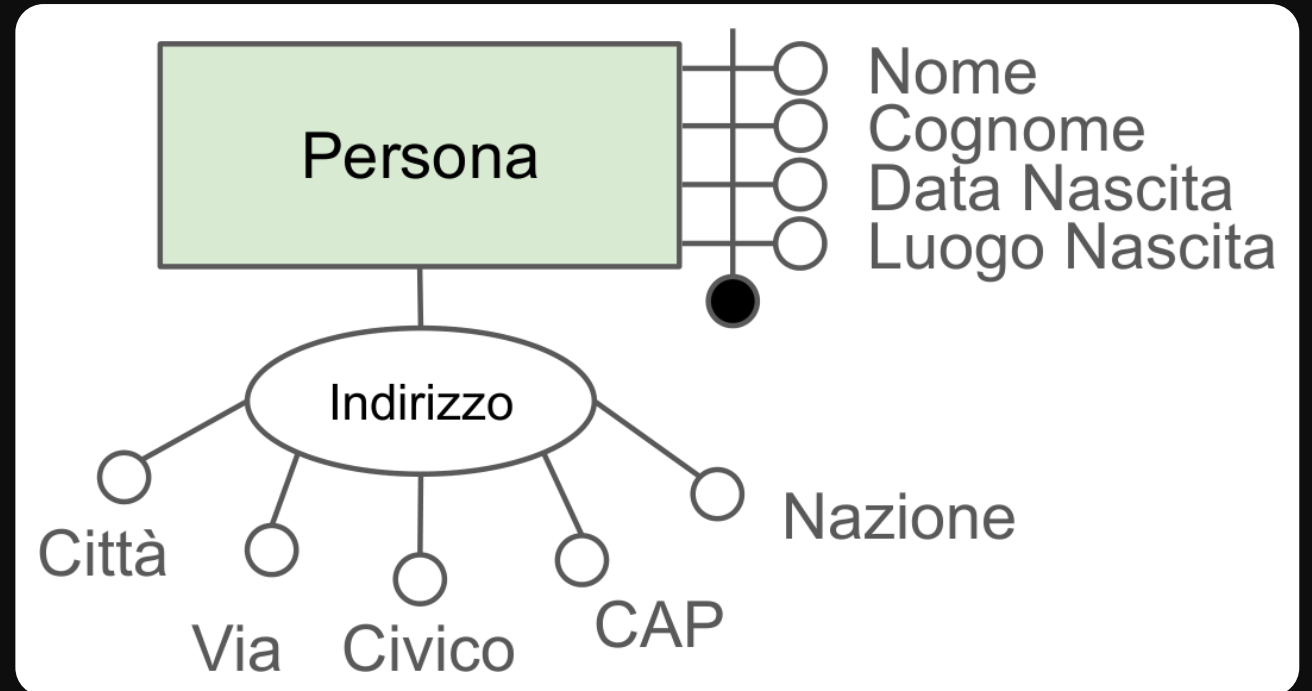
Se la chiave è **composta** da **più attributi**, potrebbe comunque **convenire** introdurre una **chiave artificiale!**

Attributi Composti

Attributi costituiti da due o più “**sotto-campi**”.

Ad esempio:

- l'attributo “**indirizzo**”
 - potrebbe essere una generica stringa, ma...
 - ... in realtà è composto da
 - nazione
 - città
 - cap
 - via
 - civico



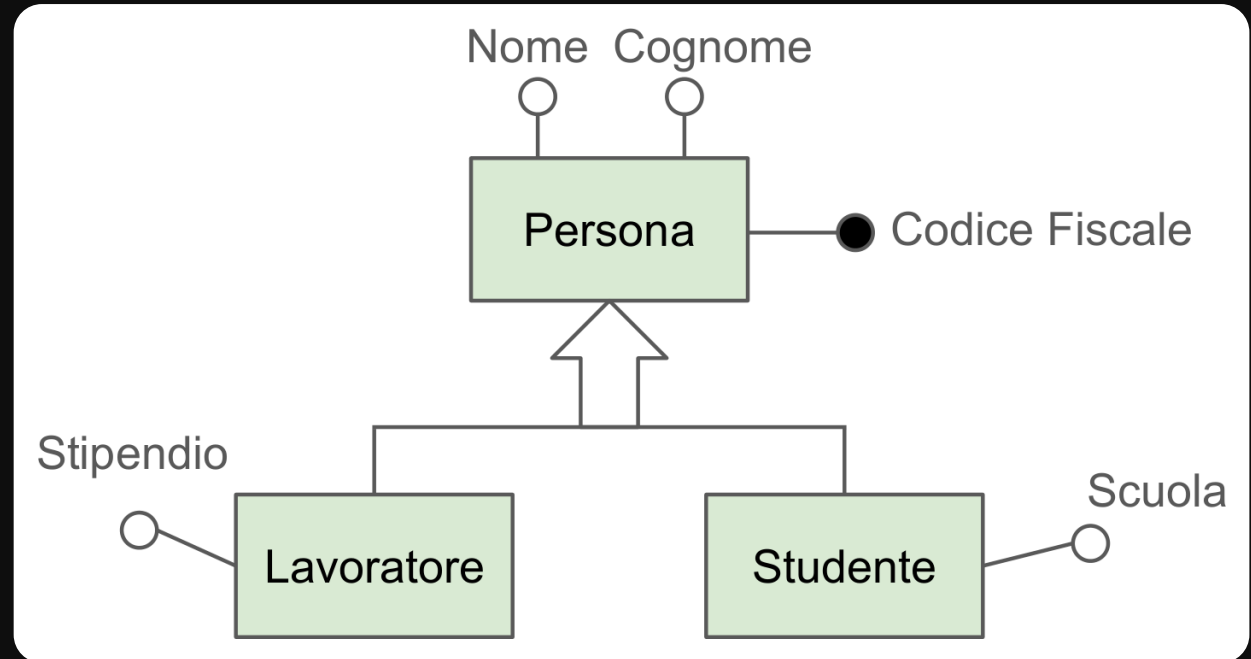
Generalizzazione/Specializzazione

Come rappresentare **entità simili**?

- concettualmente **collegate**
- con **attributi simili**
 - senza doverli duplicare ogni volta...

Analogia con **Ereditarietà** OOP!

Le **proprietà** del **genitore** sono **ereditate** dai **figli**, senza indicarle esplicitamente!

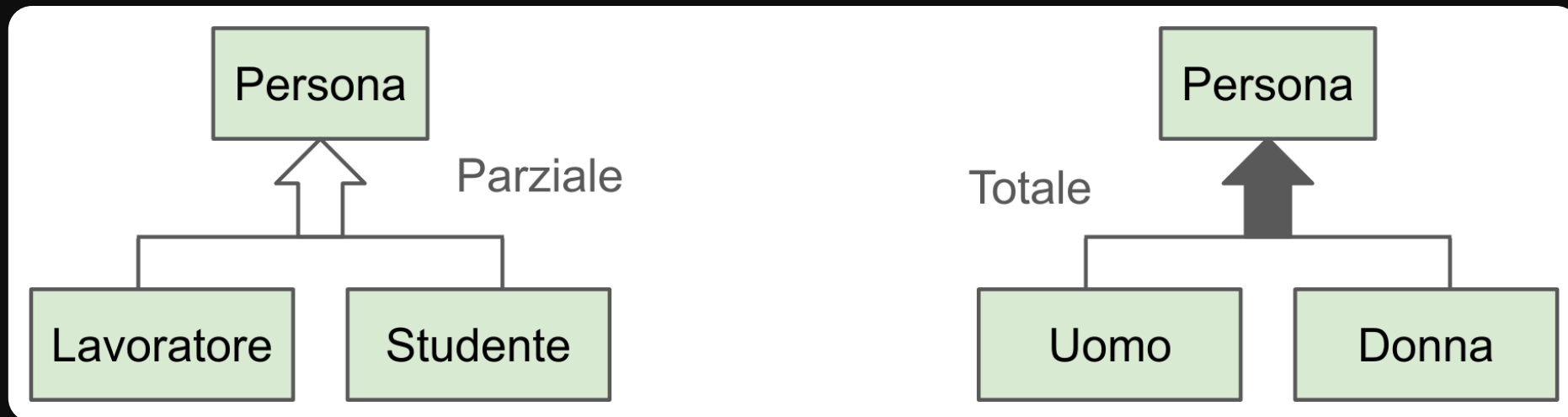


Persona **generalizza** Lavoratore e Studente
Lavoratore e Studente **specializzano** Persona

Generalizzazione/Specializzazione

Si suddividono in diverse tipologie:

- **totale/parziale:**
 - esistono occorrenze del genitore che non sono occorrenza di nessuna delle figlie?
- **esclusiva/sovrapposta:**
 - esistono occorrenze del genitore che sono occorrenze di più figlie?



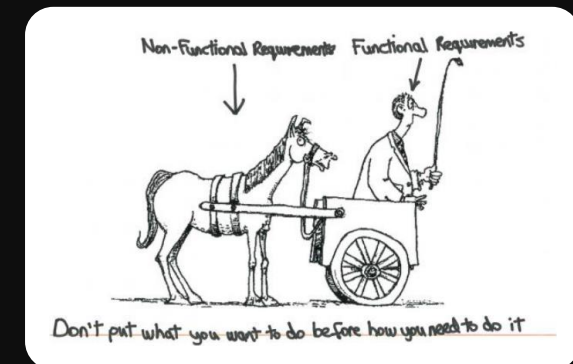
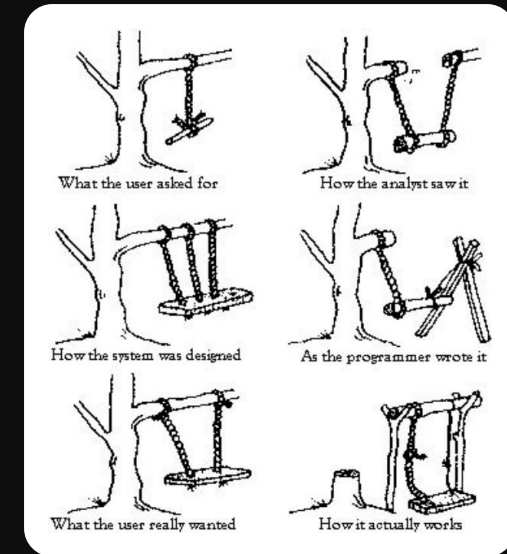
Analisi e ipotesi aggiuntive

La **descrizione** di un certo **dominio** di applicazione non è **mai chiara ed esaustiva** al 100%:

- **ipotesi aggiuntive**
 - scelte di **progettazione** per **colmare** eventuali “buchi”

Quindi è **necessaria** la fase di **analisi**:

- non è una “ripetizione” dello schema concettuale
- è la **descrizione motivata delle scelte di progettazione non scontate!**



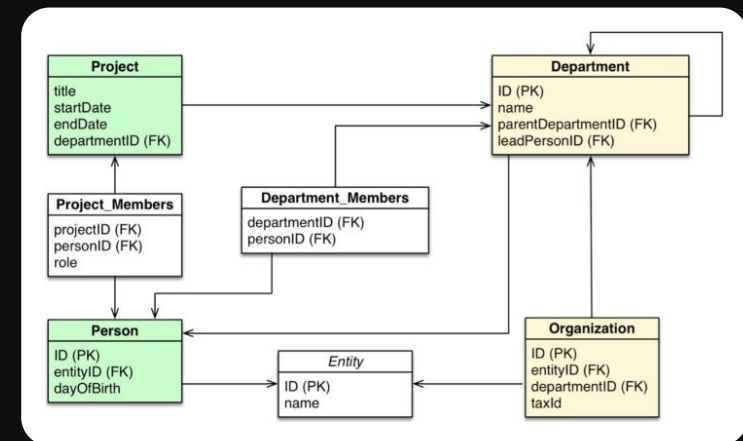
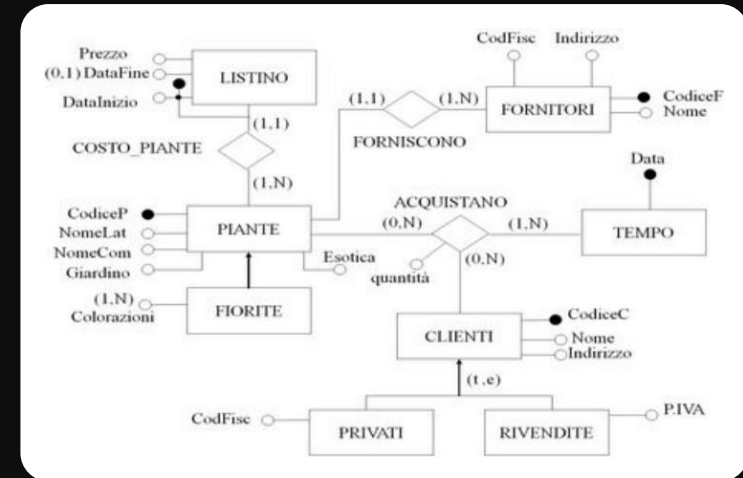
03

Progettazione Logica

Schema Logico

Il modello concettuale (schema ER) **non** è pensato per **essere implementato** in una base di dati reale.

Progettazione Logica: **costruzione** di uno schema logico che **descriva** in modo **corretto** ed **efficiente** tutte le **informazioni** presenti nello schema concettuale.



In sintesi, **convertire** lo **schema ER** in **relazioni**!

Da schema concettuale a logico

Primi step della conversione da schema ER a logico:

- analisi delle **ridondanze**
 - rimuovere componenti superflue dallo schema ER
- eliminazione delle **generalizzazioni/specializzazioni**
 - sostituirle con associazioni o accorparle nell'entità madre
- **partizionamento o accorpamento di entità o associazioni**
 - le **entità** diventano **relazioni** (tabelle) con stessi **attributi**
 - le **associazioni** diventano
 - o attributi **chiave esterna** (Foreign Key)
 - o **relazioni/tabelle**
 - **chiave** composta da gli attributi **chiave** delle **entità** coinvolte nell'associazione

Rappresentazione dello schema logico

Come si possono esprimere tabelle in forma compatta?

```
<NomeRelazione>(<Attr1>[:<Tipo1>],...,<AttrN>[:<TipoN>])
```

Ad esempio:

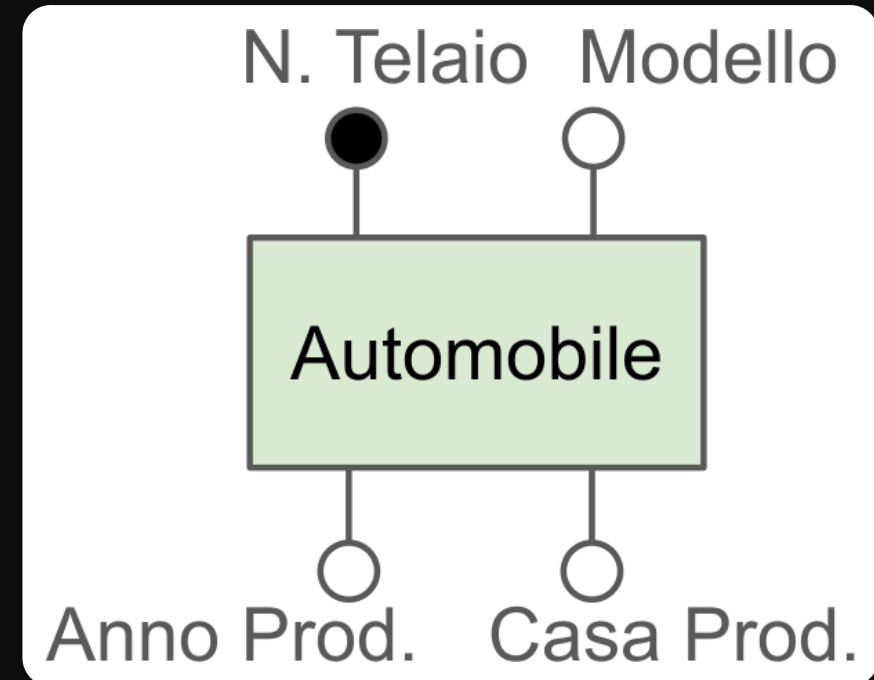
- in forma estesa
 - **Persona**(cognome:Stringa, nome:Stringa, età:Intero, sesso:Booleano)
- in forma compatta (tipo sottointeso)
 - **Persona**(cognome, nome, età, sesso)

Per indicare eventuali attributi chiave si usano dei modificatori:

- PK o sottolineatura **piena** (Primary Key)
- FK o sottolineatura **tratteggiata** (Foreign Key)

Traduzione delle entità

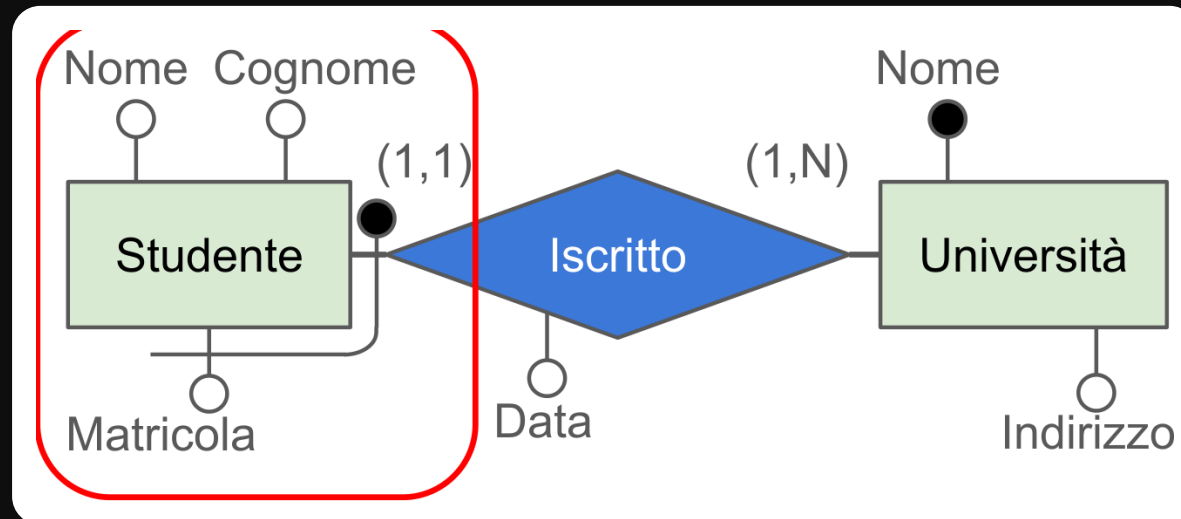
- Le **entità** diventano **relazioni**
- Gli **attributi** dell'entità diventano attributi della **relazione**
 - le **chiavi** dell'entità diventano **chiave primaria** della relazione



Automobile(PK(n_telaio), modello, anno_prod, casa_prod)

Traduzione delle entità (chiave con identificatore esterno)

In **caso** di entità con **identificatore esterno**, la **chiave primaria** sarà **composta** anche dalla **chiave** dell'altra relazione coinvolta nell'associazione.



Studente(PK(matricola), PK(FK(nome_università)), nome, cognome)

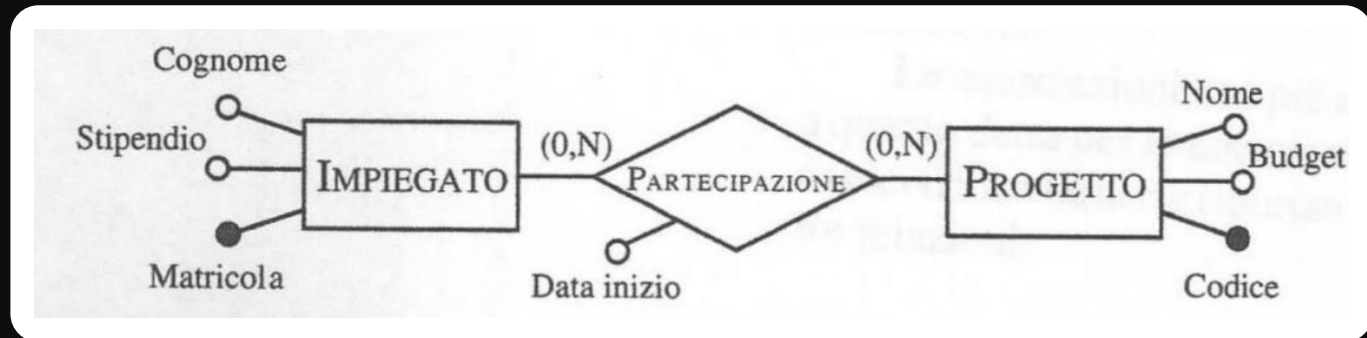
Università(PK(nome), indirizzo)

Traduzione delle associazioni

In **generale** possono essere tradotte con una **relazione** che abbia come **attributi**:

- gli **attributi dell'associazione**
- una **chiave primaria** composta da **attributi (FK)** collegati alle chiavi delle **relazioni coinvolte**

Il caso N-N è quello più generale!



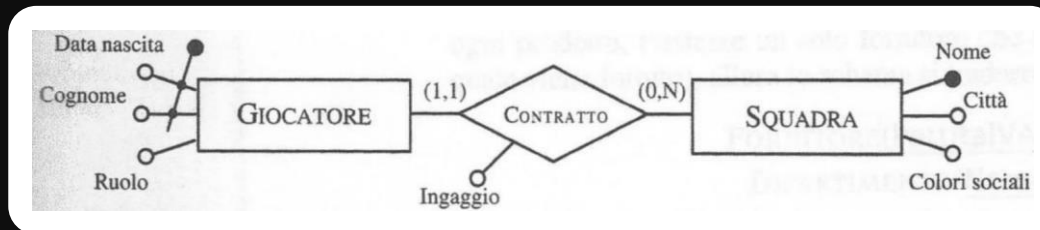
Impiegato(PK(matricola), cognome, stipendio)

Progetto(PK(codice), nome, budget)

Partecipazione(PK(FK(matricola_imp)), PK(FK(codice_prog)), data_inizio)

Traduzione delle associazioni

In casi più specifici (1-N) l'associazione può essere accorpata.



“nome_sq” non serve che sia PK dell’associazione! La coppia “data_nascita” e “cognome” è già univoca (cardinalità 1-1)

Giocatore(PK(cognome), PK(data_nascita), ruolo)

Squadra(PK(nome), città, colori_sociali)

Contratto(PK(FK(cognome_gioc)), PK(FK(data_nasc)), FK(nome_sq), ingaggio)

↓ Si può **accorpare** l’associazione nell’entità dal lato 1-1

Giocatore(PK(cognome), PK(data_nascita), ruolo, FK(nome_sq), ingaggio)

Squadra(PK(nome), città, colori_sociali)

Traduzione delle associazioni

Accorpamenti ancora più estesi sono possibili per associazioni (1-1).



1. Accorpamento su entità di Sx

Direttore(PK(codice), cognome, stipendio, FK(nome_dip), data_inizio)

Dipartimento(PK(nome), telefono, sede)

2. Accorpamento su entità di Dx

Direttore(PK(codice), cognome, stipendio)

Dipartimento(PK(nome), telefono, sede, FK(codice_dir), data_inizio)

3. Accorpamento “totale”

a. si crea un’entità unica!

Sconsigliata!

Direttore(PK(codice), cognome, stipendio, data_inizio, nome_dip, telefono_dip, sede_dip)

Traduzione delle associazioni (partecipazione opzionale)

Come accorpare l'associazione quando il lato "ad 1" ha **partecipazione opzionale** (ad es. (0,1)-(1,N) o (0,1)-(1,1) o (0,1)-(0,1))?

Potrebbe succedere che fondendo l'associazione, alcune **tuple non abbiano** una **corrispondenza** con **tuple dell'altra** relazione!

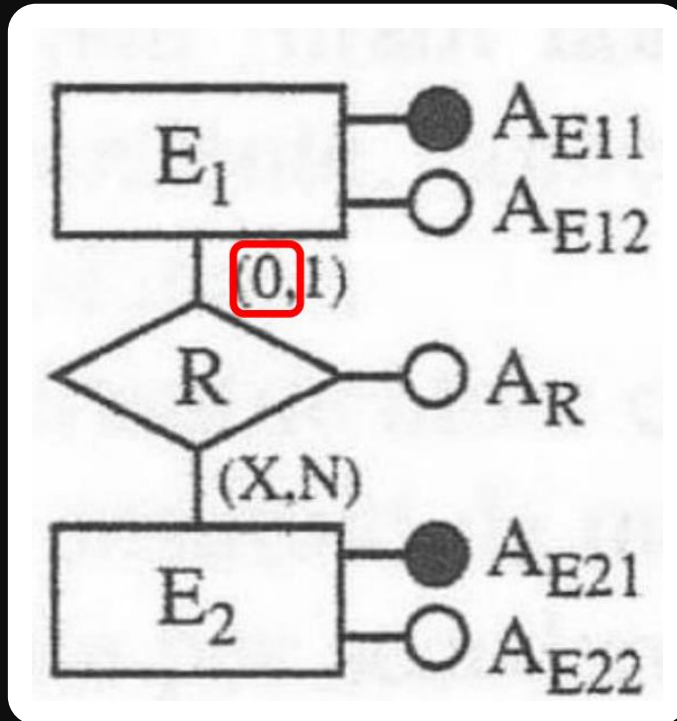
Problema: Come gestire **l'assenza** di **collegamenti** da una tupla all'altra?

- Soluzione 0: se possibile scegliere il lato (1-1)!
- Soluzione 1:
 - si **mantiene** lo **schema** di **traduzione** visto finora, ma si aggiungono **attributi** che possono avere **valori "NULL"** (indicati con *)
- Soluzione 2:
 - si **traducono** le **associazioni** come nel caso **N-N**.
 - in caso di **collegamento mancante**, semplicemente **non comparirà** una **tupla** nella **relazione derivata** dall'associazione

Traduzione delle associazioni (partecipazione opzionale)

Vantaggi e svantaggi della Soluzione 1:

- Contro: tanti valori **NULL** che “sporcano” il DB
- Pro: Il DB resta più **leggero** (ho meno tabelle)



Soluzione 1:

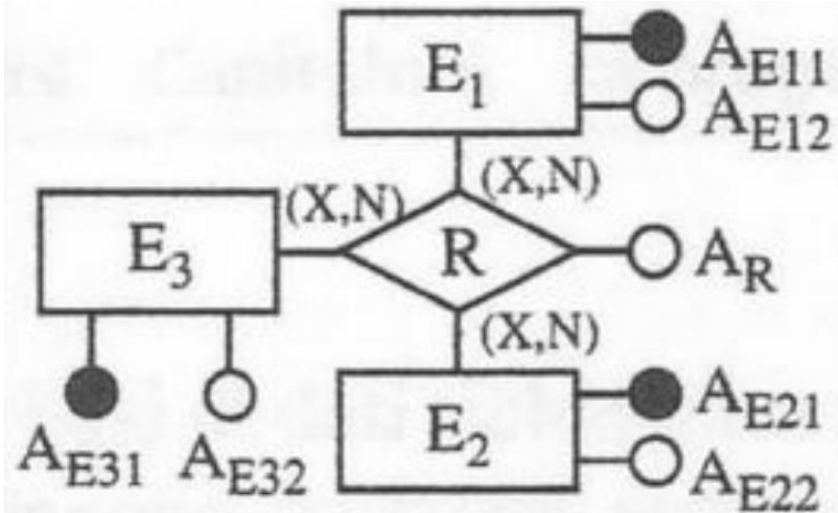
```
E1(PK(a_e11), a_e12, FK(a_e21)*, a_r*)
E2(PK(a_e21), a_e22)
```

Soluzione 2:

```
E1(PK(a_e11), a_e12)
E2(PK(a_e21), a_e22)
R(PK(FK(a_e11)), FK(a_e21), a_r)
```

Traduzione delle associazioni ternarie

Le associazioni **ternarie molti a molti** possono essere tradotte come il caso binario, creando un'**apposita relazione** derivata dall'associazione.



E1(PK(a_e11), a_e12)

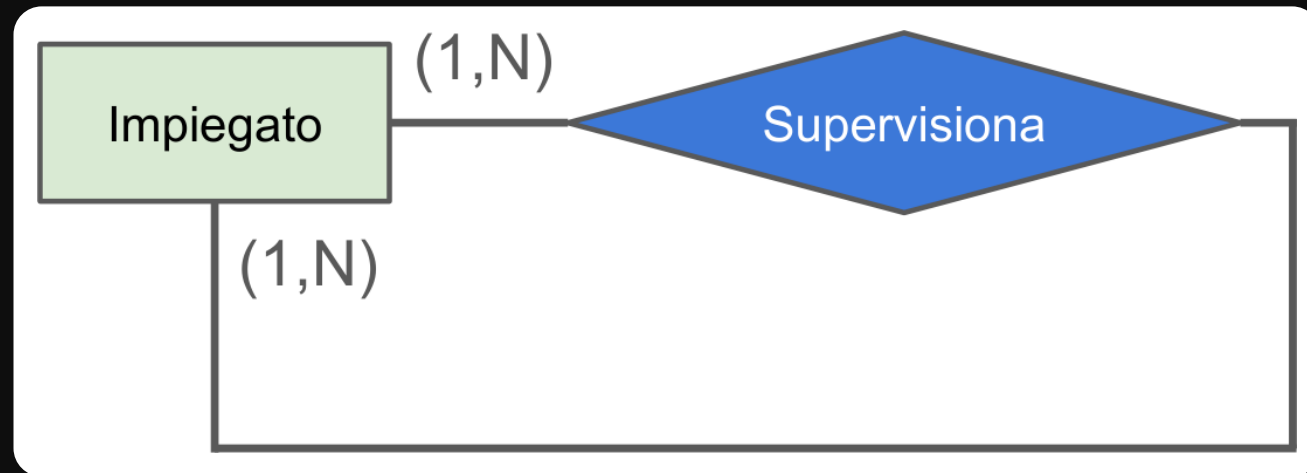
E2(PK(a_e21), a_e22)

E3(PK(a_e31), a_e32)

R(PK(FK(a_e11)), PK(FK(a_e21)), PK(FK(a_e31)), a_r)

Traduzione delle associazioni ricorsive

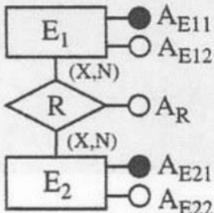
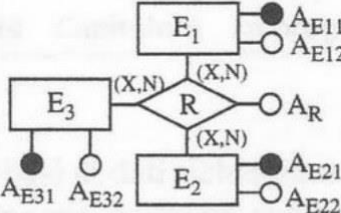
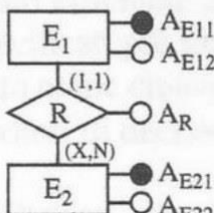
Si possono tradurre con una **relazione dedicata** anche le associazioni ricorsive, usando due chiavi esterne come predecessore e successore.

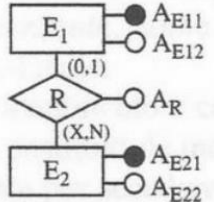
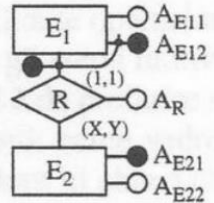


Impiegato(PK(id), nome, cognome)

Supervisione(PK(FK(id_supervisore)), PK(FK(id_supervisionato)), data_inizio)

Quadro Riassuntivo Traduzione Associazioni

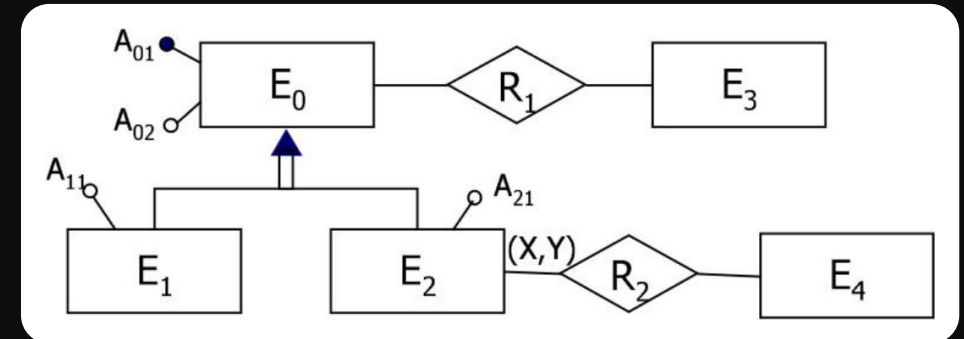
Tipologia	Concetto iniziale	Risultati possibili
Associazione binaria molti a molti		$\begin{array}{l} E_1(\underline{A_{E11}}, A_{E12}) \\ E_2(\underline{A_{E21}}, A_{E22}) \\ R(\underline{A_{E11}}, \underline{A_{E21}}, A_R) \end{array}$
Associazione ternaria molti a molti		$\begin{array}{l} E_1(\underline{A_{E11}}, A_{E12}) \\ E_2(\underline{A_{E21}}, A_{E22}) \\ E_3(\underline{A_{E31}}, A_{E32}) \\ R(\underline{A_{E11}}, \underline{A_{E21}}, \underline{A_{E31}}, A_R) \end{array}$
Associazione uno a molti con partecipazione obbligatoria		$\begin{array}{l} E_1(\underline{A_{E11}}, A_{E12}, A_{E21}, A_R) \\ E_2(\underline{A_{E21}}, A_{E22}) \end{array}$

Associazione uno a molti con partecipazione opzionale		$\begin{array}{l} E_1(\underline{A_{E11}}, A_{E12}) \\ E_2(\underline{A_{E21}}, A_{E22}) \\ R(\underline{A_{E11}}, A_{E21}, A_R) \end{array}$ Oppure: $\begin{array}{l} E_1(\underline{A_{E11}}, A_{E12}, A_{E21}^*, A_R^*) \\ E_2(\underline{A_{E21}}, A_{E22}) \end{array}$
Associazione con identificatore esterno		$\begin{array}{l} E_1(\underline{A_{E12}}, \underline{A_{E21}}, A_{E11}, A_R) \\ E_2(\underline{A_{E21}}, A_{E22}) \end{array}$

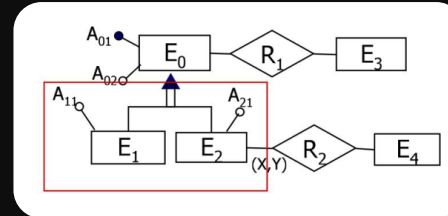
Traduzione delle generalizzazioni/specializzazioni

In generale non esiste un'unica strategia, ma bisogna valutare i casi specifici:

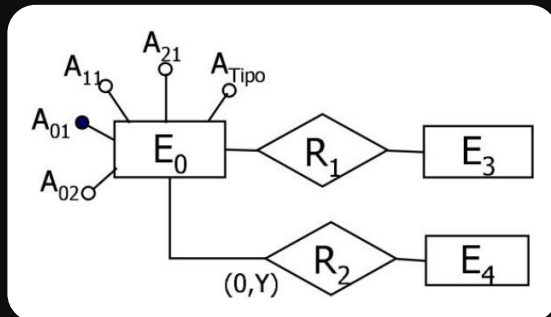
- accorpare le entità **figlie** nel **genitore**
 - le entità figlie non hanno associazioni mentre il genitore sì
- accorpare il **genitore** nelle **figlie**
 - il genitore non ha associazioni mentre le figlie sì
- **sostituire** con **associazioni** e tradurre con le tecniche già viste
 - sia il genitore che le entità figlie hanno associazioni specifiche
- soluzioni “**miste**”
 - combinazione delle precedenti



Traduzione delle generalizzazioni/specializzazioni

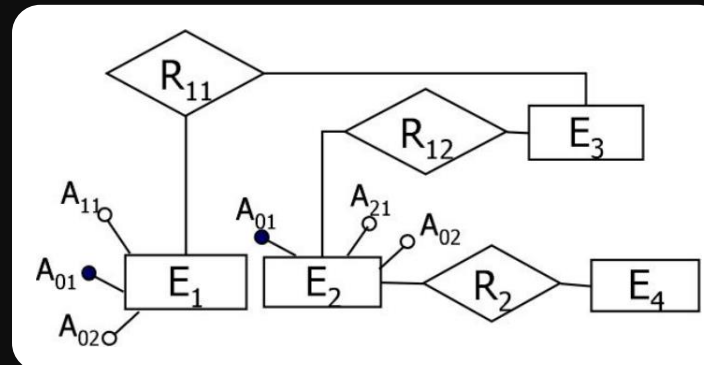


Accorpamento nel **genitore**



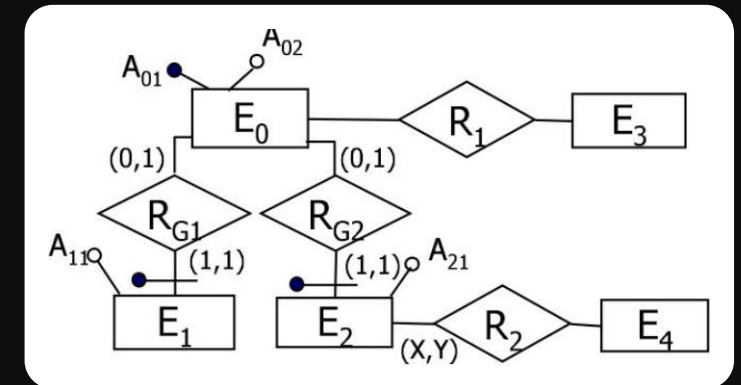
- attributi figlie nullable
- aggiunta attributo “tipo” (distingue la figlia)
- card. minima dell’associazione figlia=0

Accorpamento nelle **figlie**



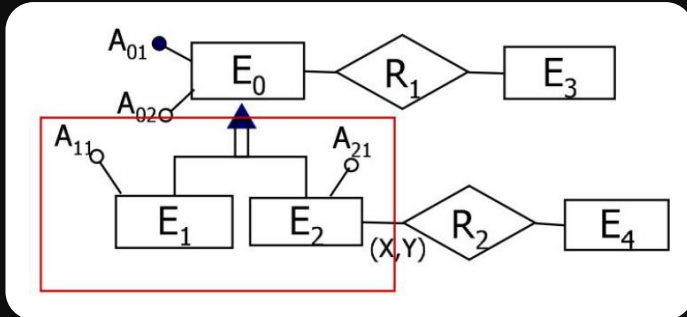
- ogni associazione sul padre è ripetuta per ogni figlia

Sostituzione con **associazioni**



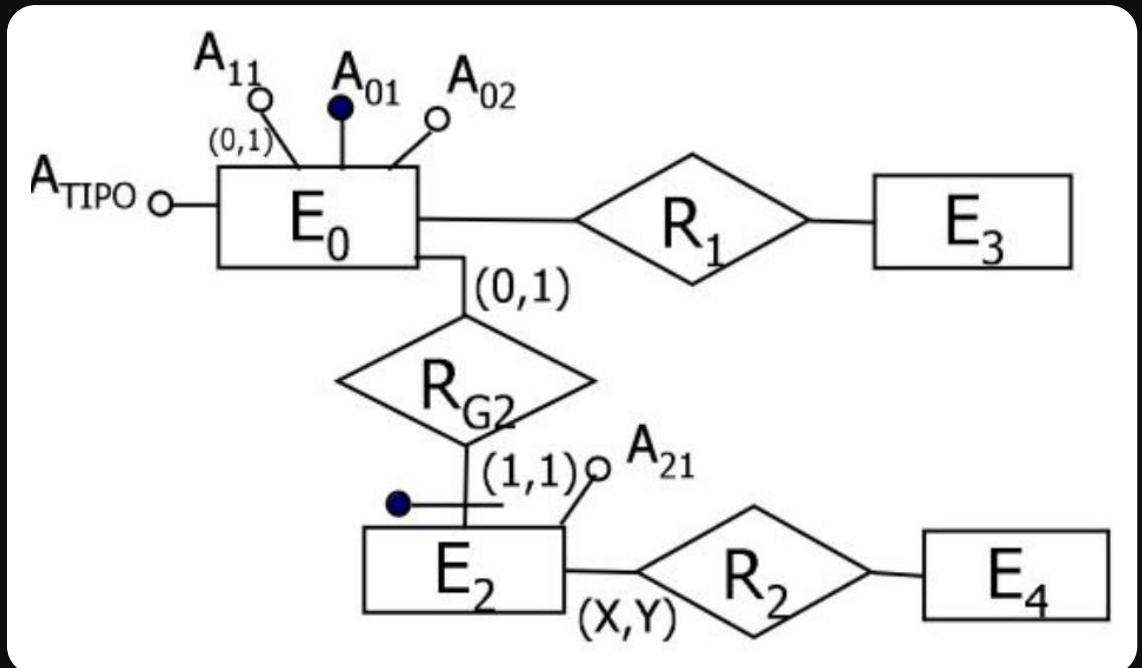
- ogni istanza del padre può partecipare solo ad una associazione con le figlie (card. min. = 0)

Traduzione delle generalizzazioni/specializzazioni



- si accorpa soltanto E1 perchè non ha associazioni
- si usa l'attributo "Atipo" per distinguere istanze di E1 da quelle di E0
- l'altra specializzazione diventa un'associazione
- la PK di E2 è anche una FK collegata ad A01 di E0.

Soluzione "mista"



Vincolo di Integrità referenziale

Le **chiavi esterne** (FK) consentono a **tuple** di diverse relazioni di essere **collegate**.

Tuttavia, i campi FK **non possono** assumere **qualsiasi valore**!

- deve esserci **corrispondenza** con uno dei **valori** della **PK** a cui fa **riferimento**

Limitare i **valori** assunti dai campi FK ai soli valori possibili della PK serve a **garantire l'integrità** dei **riferimenti**.

Ordini			
Cliente	NumOrdine	Data	Importo
3145	321	10/12/2019	€ 230,00
1231	322	11/12/2019	€ 349,00
1231	323	14/12/2019	€ 451,00
1127	324	17/12/2019	€ 230,00
4871	325	18/12/2019	€ 273,00

violazione del vincolo di integrità referenziale

Clienti			
CodiceCliente	Cognome	Nome	Città
1127	Rossi	Mario	Roma
1231	Bianchi	Giuseppe	Milano
3145	Verdi	Giovanni	Napoli

Vincolo di integrità referenziale: mantiene la **coerenza** delle **associazioni**

04

Normalizzazione

Normalizzazione

Una volta realizzato lo **schema logico**, come ci si può assicurare che sia **ben costruito**?

Esistono dei **criteri** per “**certificare**” la **qualità** di una base di dati?

Sì! Le cosiddette “**Forme Normali**” (NF).

- 1NF
 - esistenza della PK e atomicità degli attributi
- 2NF
 - ogni attributo dipende da tutta la PK
- 3NF
 - assenza dipendenze transitive tra attributi non PK
- 4NF... (di solito ci si ferma alla 3NF)

Se lo schema ER è **ben progettato**, lo schema logico potrebbe **già rispettare** le 3 Forme Normali!

Tuttavia, va verificato!

Ristrutturazione/Decomposizione

Se un DB non rispetta le 3 forme normali, lo si deve **ristrutturare**:

- **decomponendo** le relazioni così da **eliminare** le **ridondanze**

Tale ristrutturazione deve avvenire rispettando **due criteri**:

- **conservazione delle dipendenze**
 - mantenimento dei vincoli di integrità
- **assenza di perdite**
 - devo poter ricostruire tutte le informazioni originarie

Non rispettare le forme normali significa andare incontro a

- **ridondanze** che causano **anomalie** nell'utilizzo del DB!
- **dimensioni** e **velocità** di accesso **poco ottimizzati**

Esempio relazione non normalizzata e anomalie

Cognome	Nome	Funzione	Stipendio	Scuola
Rossi	Mario	Preside	50 €	ITIS G. Verdi
Verdi	Filippo	Docente	35 €	ITIS G. Verdi
Verdi	Filippo	Docente	35 €	ITIS P. Rossi
Gialli	Enrico	Docente	35 €	ITIS G. Verdi
Paperino	Paolino	Docente	30 €	ITIS G. Verdi
Paperino	Paolino	Docente	30 €	ITIS S. Gialli
Ali	Baba	Bidello	16 €	ITIS G. Verdi
Verdi	Giuseppe	Bidello	16 €	ITIS G. Verdi

Problemi:

- **ridondanze** (valori ripetuti inutilmente tante volte)...
- ...che **causano anomalie** di modifica, cancellazione, inserimento

Esempio relazione non normalizzata e anomalie

Cognome	Nome	Funzione	Stipendio	Scuola
Rossi	Mario	Preside	50 €	ITIS G. Verdi
Verdi	Filippo	Docente	35 €	ITIS G. Verdi
Verdi	Filippo	Docente	35 €	ITIS P. Rossi
Gialli	Enrico	Docente	35 €	ITIS G. Verdi
Paperino	Paolino	Docente	30 €	ITIS G. Verdi
Paperino	Paolino	Docente	30 €	ITIS S. Gialli
Ali	Baba	Bidello	16 €	ITIS G. Verdi
Verdi	Giuseppe	Bidello	16 €	ITIS G. Verdi

Anomalie di

- **modifica**
 - ad esempio, se uno stipendio cambia lo devo modificare in tutte le tuple!
- **cancellazione**
 - ad esempio, se un dipendente resta senza scuola deve essere cancellato!
- **inserimento**
 - ad esempio, un nuovo dipendente non assegnato ad una scuola non può essere inserito!

1° Forma Normale

Un DB è in 1NF se per **ogni relazione** valgono le seguenti regole:

- **esistenza** della **chiave** primaria
 - insieme di attributi che identifica univocamente ogni tupla
- **atomicità** degli **attributi**
 - il dominio di ciascun elemento delle tuple ha valori atomici
 - no attributi “composti” (esempio classico: indirizzo)

Students

FirstName	LastName	Knowledge
Thomas	Mueller	Java, C++, PHP
Ursula	Meier	PHP, Java
Igor	Mueller	C++, Java

- Qual è la **PK**?
 - usare **attributi** già **presenti**
 - introdurre **chiave artificiale**
- L'attributo “Knowledge” non è **atomico**!
 - **accorpare** tutto in una **stringa**
 - **creare** una **tupla** per ogni **valore**
 - **creare** un **attributo** per ogni **valore**
 - **creare** una **relazione** apposita

Codice Fiscale	Cognome	Nome	Residenza
LBRRSS97Y12T344A	Rossi	Alberto	Via Roma, 22; 22100 Como
GNNBNCT99A11L611B	Bianchi	Gianni	Piazza Lodi, 12; 80010 Napoli

Dipendenza Funzionale

Dati due **attributi X** e **Y** di una **relazione R**:

- si dice che **Y dipende funzionalmente** da **X** se **ogni valore** di **X** in **R** viene **associato** con un **solo valore** di **Y**.
 - in questo caso, **X** si dice **determinante** per **Y**.

ID_Anag	Cognome	Nome	Indirizzo	Località	CAP	Telefono
1	Rossi	Mario	Via Lombardia, 1	Como	22100	031.123123
2	Verdi	Filippo	Via Marche, 34	Milano	20100	02.321321
3	Gialli	Enrico	Viale Abruzzi, 4	Torino	10100	011.132132
4	Paperino	Paolino	Via Lazio, 34	Como	22100	031.121212
5	Ali	Baba	Via Puglia, 34	Milano	20100	02.332233
6	Verdi	Luca	Via Emilia, 32	Como	22100	031.112233

Quali sono le **dipendenze funzionali** in questa relazione (anagrafiche)?

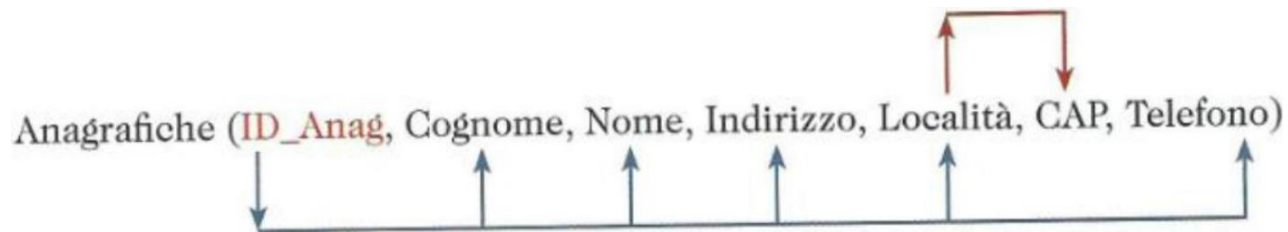
Dipendenza Funzionale

ID_Anag	Cognome	Nome	Indirizzo	Località	CAP	Telefono
1	Rossi	Mario	Via Lombardia, 1	Como	22100	031.123123
2	Verdi	Filippo	Via Marche, 34	Milano	20100	02.321321
3	Gialli	Enrico	Viale Abruzzi, 4	Torino	10100	011.132132
4	Paperino	Paolino	Via Lazio, 34	Como	22100	031.121212
5	Ali	Baba	Via Puglia, 34	Milano	20100	02.332233
6	Verdi	Luca	Via Emilia, 32	Como	22100	031.112233

Se uno o più attributi sono **chiave**, tutti gli altri **dipendono funzionalmente** da esso/i!

Tutti gli attributi **dipendono funzionalmente** dalla **chiave**, ad eccezione del **CAP**!

- “località” è **determinante** per “CAP”
- ogni **valore** di “CAP” è associato ad **un solo valore** di “località”



Notazione: $R.x \rightarrow R.y$
 “y” dipende da “x”

Dipendenza Funzionale

Possiamo quindi definire la normalizzazione come un processo di “**decomposizione**” di **relazioni grandi** in relazioni più **piccole**

- con l’**obiettivo** di **rimuovere** le **ridondanze** (eliminando le **anomalie**)
- creando **relazioni** i cui **attributi non chiave** che **dipendono totalmente** ed **esclusivamente** dalla **chiave**

Quindi, come si può **decomporre** la tabella precedente?

- **eliminando** l’attributo CAP (o località)
- **creando** una **relazione** a parte che memorizzi CAP e località

2° Forma Normale

Un DB è in 2NF se per **ogni relazione** valgono le seguenti **regole**:

- vale già la **1NF**
- **tutti gli attributi non chiave** devono dipendere dall'insieme di **tutti** gli attributi **chiave**
 - non esistono dipendenze parziali dalla chiave

ID_Fornitore	Stato	Città	ID_Articolo	Qtà
S1	20	Roma	P1	300
S1	20	Roma	P2	400
S1	20	Roma	P3	200
S1	20	Roma	P4	150
S1	20	Roma	P5	350
S1	20	Roma	P6	500
S2	10	Londra	P1	400
S2	10	Londra	P2	300
S3	30	Parigi	P2	200
S4	40	Budapest	P2	100
S4	40	Budapest	P4	150
S4	40	Budapest	P5	250

PK: ID_Fornitore, ID_Articolo

Dipendenze Funzionali:

ID_Fornitore → Città, Stato
 Città → Stato
 (ID_Fornitore, ID_Articolo) → Qtà

2NF non rispettata!

2° Forma Normale (esempio 1)

Una volta identificati i **determinanti**, la relazione può essere **decomposta**:

- creando una **nuova tabella** per ogni attributo **determinante** (sottoinsieme della PK)
- **spostando** le **colonne** dipendenti da esso dalla relazione di partenza alla **nuova**
- eventuali **FK** si possono **mantenere**

Fornitori		
ID_Fornitore(pk)	Stato	Città
S1	20	Roma
S2	10	Londra
S3	30	Parigi
S4	40	Budapest

Tabella 1		
ID_Articolo(pk)	ID_Fornitore(pk)(fk)	Qtà
P1	S1	300
P2	S1	400
P3	S1	200
P4	S1	150
P5	S1	350
P6	S1	500
P1	S2	400
P2	S2	300
P2	S3	200
P2	S4	100
P4	S4	150
P5	S4	250

2° Forma Normale (esempio 2)

Matricola	Codice Materia	Alunno	Voto Materia
1234	Info1	Rossi	6
1234	Elettro2	Rossi	7
1235	Elettro2	Verdi	8

1NF rispettata: la **PK esiste** (Matricola e Codice Materia) e attributi **atomici**.

2NF non rispettata: **Alunno dipende** solo da **Matricola**

Ristrutturando la relazione di partenza:

Matricola	Codice Materia	Voto Materia
1234	Info1	6
1234	Elettro2	7
1235	Elettro2	8

Matricola	Alunno
1234	Rossi
1235	Verdi

Osservazione: se ho PK composte da **un solo attributo**, la **2NF** è sempre **rispettata**!

3° Forma Normale

Un DB è in 3NF se per **ogni relazione** valgono le seguenti regole:

- vale già la **2NF**
- gli attributi **non chiave** possono **dipendere** soltanto dalla **chiave**
 - non esistono dipendenze **transitive**

Anomalie:

- non posso **inserire** una **città** finché non c'è un **fornitore**
- **cancellare** un **fornitore** cancella anche la **città**

Tabella 1		
ID_Articolo(pk)	ID_Fornitore(pk)(fk)	Qtà
P1	S1	300
P2	S1	400
P3	S1	200
P4	S1	150
P5	S1	350
P6	S1	500
P1	S2	400
P2	S2	300
P2	S3	200
P2	S4	100
P4	S4	150
P5	S4	250

Fornitori		
ID_Fornitore(pk)	Stato	Città
S1	20	Roma
S2	10	Londra
S3	30	Parigi
S4	40	Budapest

3° Forma Normale

- “Tabella 1” è in **3NF** (qtà ha dipendenza diretta e totale dalla PK)
- “Fornitori” **non** è in **3NF**
 - “**stato**” dipende da “**città**”
 - “**città**” dipende dalla **PK**
 - Dipendenza **Transitiva!**

Le dipendenze **transitive non** vengono intercettate dalla **2NF!**

- Infatti, la **dipendenza** dalla PK in questo caso **non** è **parziale**, ma **indiretta**

Tabella 1		
ID_Articolo(pk)	ID_Fornitore(pk)(fk)	Qtà
P1	S1	300
P2	S1	400
P3	S1	200
P4	S1	150
P5	S1	350
P6	S1	500
P1	S2	400
P2	S2	300
P2	S3	200
P2	S4	100
P4	S4	150
P5	S4	250

Fornitori		
ID_Fornitore(pk)	Stato	Città
S1	20	Roma
S2	10	Londra
S3	30	Parigi
S4	40	Budapest

3° Forma Normale (esempio 1)

Occorre **decomporre** “Fornitori” (procedura analoga a quella della 2NF):

Fornitori		
ID_Fornitore(pk)	Stato	Città
S1	20	Roma
S2	10	Londra
S3	30	Parigi
S4	40	Budapest



Id_Fornitore()	Città(fk)
S1	Roma
S2	Londra
S3	Parigi
S4	Budapest

Città(pk)	Stato
Roma	20
Londra	10
Parigi	30
Budapest	20

3° Forma Normale (esempio 2)

Codice Fiscale	Corso	Istruttore
LBRRSS93Y12T344A	Body building	Marco
GNNBNCT96A11L611B	Body building	Marco
LBRMNN96E64A112A	Body building	Marco
GLSTMT99U66P109B	Fitness	Federica



Codice Fiscale	Corso
LBRRSS93Y12T344A	Body building
GNNBNCT96A11L611B	Body building
LBRMNN96E64A112A	Body building
GLSTMT99U66P109B	Fitness

Corso	Istruttore
Body building	Marco
Fitness	Federica

Utilizzo dello schema ER

Presa una relazione **non normalizzata**, una volta **compreso** il **dominio** di **interesse**, è possibile **definire** uno **schema ER** che la rappresenti, ricavando uno schema concettuale che comprenda **tutte** le **informazioni** di tale relazione.

Si identificano quindi:

- entità
- associazioni
- attributi
- cardinalità

Si può quindi **procedere** alla **traduzione** in schema **logico**, ottenendo così una **rappresentazione** che rispetta la **3NF**!

Esercizi

JobID	JobDate Time	driverID	driver Name	taxiID	clientID	clientName	jobPickUpAddress
1	25/07/14 10.00	D1	Joe Bull	T1	C1	Anne Woo	1 Storrie Rd, Paisley
2	29/07/14 10.00	D1	Joe Bull	T1	C1	Anne Woo	1 Storrie Rd, Paisley
3	30/07/14 11.00	D2	Tom Win	T2	C1	Anne Woo	3 High Street, Paisley
4	2/08/14 13.00	D3	Jim Jones	T3	C2	Mark Tin	1A Lady Lane, Paisley
5	2/08/14 13.00	D4	Steven Win	T1	C3	John Seal	22 Red Road, Paisley
6	25/08/14 10.00	D2	Tom Win	T2	C4	Karen Bow	17 High Street, Paisley

studentID	studentName	courseID	courseTitle	credits	grade	semesterID	semesterPeriod
S103	James Bradely	CS103	Data Analytics	3	B	Fall2022	Aug 25 - Dec 21
S103	James Bradely	CS395	DB Systems	3	A	Fall2022	Aug 25 - Dec 21
S105	Nancy Herman	CS395	DB Systems	3	A	Fall2022	Aug 25 - Dec 21
S103	James Bradely	CS395	DB Systems	3	F	Spring2022	Jan 24 - May 22
S103	James Bradely	MT206	Pre-calculus	4	C	Spring2022	Jan 24 - May 22
S105	Nancy Herman	MT206	Pre-calculus	4	B	Spring2022	Jan 24 - May 22
S105	Nancy Herman	CS103	Data Analytics	3	A	Spring2022	Jan 24 - May 22

Esercizi

A partire dalle seguenti relazioni, e sulla base dei nomi scelti per essere e per i relativi attributi, normalizzare le seguenti relazioni.

- 1 Anagrafica (Nome, Cognome, Indirizzo, CAP, Città, Provincia, Regione, Telefono, Nazione)
- 2 Interrogazioni (Materia, Voto, Data, Nome, Cognome, Classe, Sezione)
- 3 Professori (Cognome, Nome, Indirizzo, Città, CAP, Classe, Sezione, Materia)
- 4 CDMusicali (Gruppo, Titolo, Canzone1, Canzone2, ... CanzoneN, Genere, Lingua)
- 5 Fatture (Cliente, Indirizzo, NrItem, NumeroFat, Data, Descrizione, Quantità, Prezzo Unitario)
- 6 Animali (CodAnagrafico, Razza, NomeProprio, Sesso, DataNascita, LuogoNascita, Nazione)
- 7 Dipendenti (Matricola, Nominativo, Stipendio, Progetto, CodBilancio, Funzione, Sede)
- 8 Fornitori (Fornitore, Indirizzo, NumeroFat, Data, Descrizione, Quantità, Prezzo Unitario)
- 9 Alunni (Cognome, Nome, Indirizzo, Città, CAP, Classe, Sezione, Specializzazione)
- 10 Gite (Classe, Sezione, Specializzazione, Periodo, Destinazione, Accompagnatore)
- 11 Automobili (Targa, Modello, CasaAutomob, Anno Immatric, Colore, Cilindrata, Categoria)
- 12 Francobolli (Nazione, Tematica, NumCatalogo, ValoreNuovo, ValoreUsato, Serie, Periodo)
- 13 Contravvenzioni (codMulta, codAgente, costoMulta, targa, codModello, descModello, codMarca, descMarca, nomeAgente, data, ora, descMulta)
- 14 Giocatori (Nazione, Squadra, Ruolo, Cognome, Nome, CittàSquadra, CittàNascita, Numero)
- 15 Libri (Autore1, Autore, ... AutoreN, Editore, Collana, Genere, AnnoPublicaz, Prezzo, Nazione, Lingua, Titolo, ISBN)

05

Introduzione al Linguaggio SQL

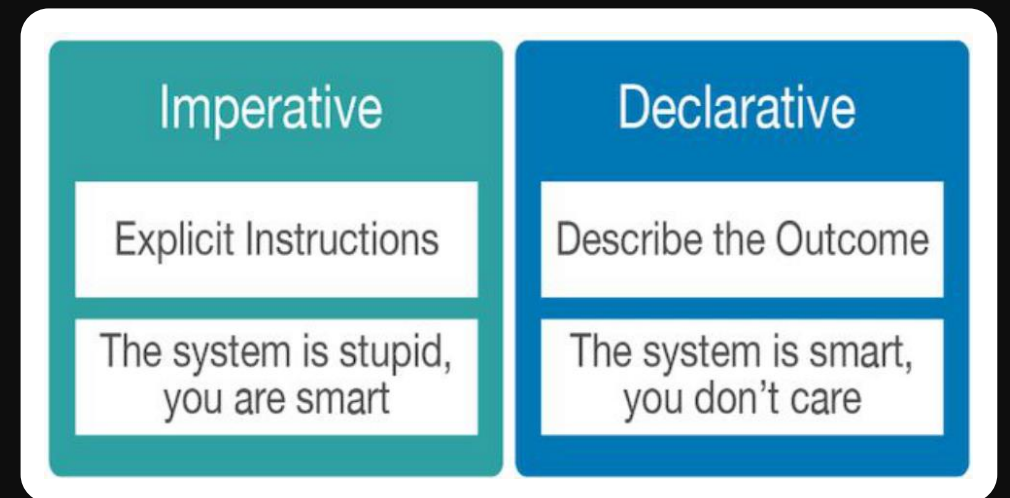
I linguaggi per i database

Nel corso degli anni, i linguaggi di programmazione di tipo **dichiarativo** sono diventati lo **standard** nei RDBMS.

- si descrive il **risultato...**
- e non **come** deve essere ottenuto!

Vantaggi:

- maggiore **leggibilità**
- **ottimizzazione** delle query
 - motore di esecuzione query che ottimizza automaticamente. non è lo sviluppatore che deve “sforzarsi” di ottimizzare la query
- maggiore **flessibilità** (non serve conoscere i dettagli interni del DBMS)



Relazioni vs Tabelle

Anche se finora i due termini sono stati usati in maniera intercambiabile, ci sono delle differenze importanti.

	Relazioni	Tabelle
Righe/Tuple Ripetute	Non permesse (relazione = insieme)	Permesse (si tende ad evitarle con PK)
Utilizzo	“Astratto” Schema logico Algebra relazionale	“Concreto” nei DBMS Schema fisico Comandi SQL

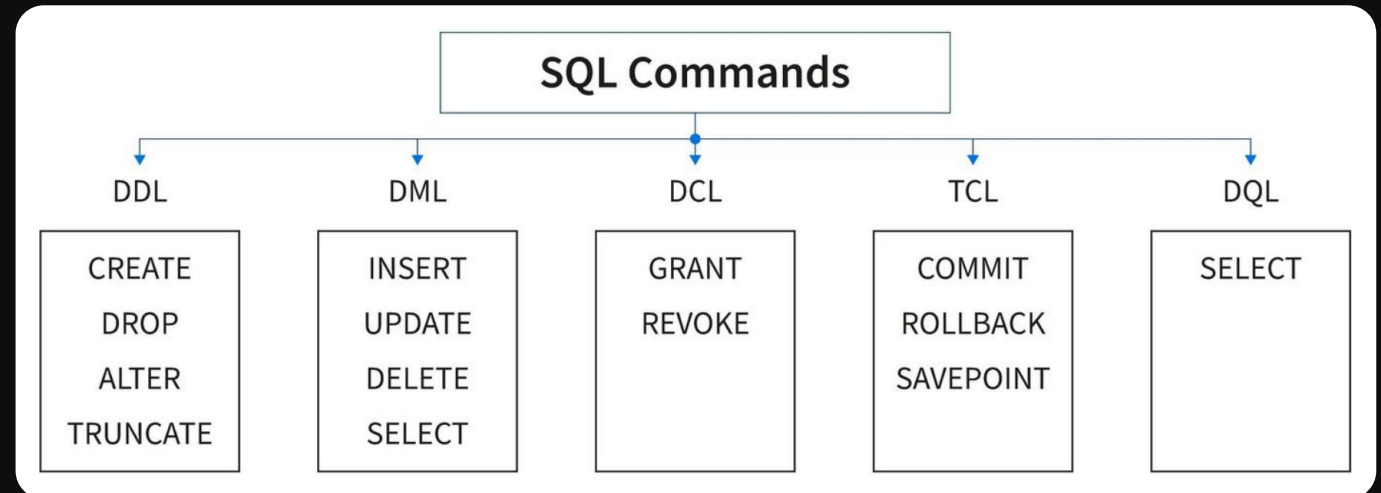
Tabella = implementazione “fisica” di una **relazione**
I **linguaggi** di programmazione nei DBMS lavorano su **tabelle**!

I linguaggi per i database

Linguaggio per RDBMS: SQL (e sue varianti/dialetti).

Esso presenta diverse “famiglie” di comandi:

- Data **D**efinition Language
- Data **M**anipulation Language
- Data **Q**uery Language, ecc...



Data Definition Language

Il DDL serve a **definire/modificare** lo **schema** della Base di Dati:

- **nomi** delle **tabelle**
- **nomi** degli **attributi**
- **tipi** degli **attributi**
- **vincoli**
 - integrità referenziale
 - comportamento in caso di
 - cancellazione
 - aggiornamento
 - lunghezza
 - autoincremento
 - nullabilità
 - unicità

Comandi principali:

- **CREATE TABLE**
 - crea una nuova tabella
- **ALTER TABLE**
 - modifica tabella esistente
- **TRUNCATE TABLE**
 - cancella le tuple di una tabella
- **DROP TABLE**
 - cancella tabella esistente

Data Definition Language (CREATE TABLE)

```
CREATE TABLE Docente (
  id_docente INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(50) NOT NULL,
  cognome VARCHAR(50) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE
);
```

```
CREATE TABLE Materia (
  id_materia INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL UNIQUE
);
```

```
CREATE TABLE Insegnamento (
  id_docente INT NOT NULL,
  id_materia INT NOT NULL,
  ore_lezione INT NOT NULL,
  PRIMARY KEY (id_docente, anno_immatricolazione),
  FOREIGN KEY (id_docente) REFERENCES Docente(id_docente)
    ON DELETE CASCADE
    ON UPDATE RESTRICT,
  FOREIGN KEY (id_materia) REFERENCES Materia(id_materia)
    ON DELETE SET NULL
    ON UPDATE CASCADE
);
```

Vincolo “in **linea**” (tra un attributo e l’altro)

ON UPDATE/DELETE: cosa succede alle **tuple** della tabella “**figlia**” quando si **elimina** una **tupla** della tabella “**padre**”?

Vincoli “in **coda**” (dopo gli attributi)

- CASCADE
 - Modifica/elimina le tuple corrispondenti
- SET NULL o DEFAULT
 - Setta a NULL o al valore default le FK corrispondenti
- RESTRICT
 - impedisce la cancellazione delle tuple corrispondenti

Data Manipulation Language (INSERT)

Il DML consente di manipolare i dati presenti nelle tabelle.

```
INSERT INTO Docente (nome, cognome, email)
VALUES ('Mario', 'Rossi', 'mariorossi@scuolaesempio.it'),
       ('Laura', 'Bianchi', 'laurabianchi@scuolaesempio.it');
```

```
INSERT INTO Materia (nome_materia)
VALUES ('Matematica'),
       ('Italiano');
```

```
INSERT INTO Insegnamento (id_docente, id_materia, ore_lezione)
VALUES (1, 1, 4),
       (2, 2, 3),
       (1, 2, 2);
```

Comandi principali:

- **INSERT INTO**
 - inserisce nuove tuple nella tabella
- **UPDATE**
 - aggiorna le tuple che soddisfano una condizione
- **DELETE**
 - cancella le tuple che soddisfano una condizione

Perché non viene specificato il valore della PK?

È auto-incrementante!

Data Manipulation Language (UPDATE e DELETE)

```
UPDATE Docente  
SET nome = 'Luca', cognome = 'Verdi'  
WHERE id_docente = 1;
```

Innesca **ON UPDATE!**

```
UPDATE Insegnamento  
SET ore_lezione = 5  
WHERE id_materia = 1;
```

- vengono **aggiornate** tutte le **tuple corrispondenti** nella “figlia”, cioè la tabella “**Insegnamento**”
- il **come** viene **deciso** nella CREATE TABLE

```
DELETE FROM Insegnamento  
WHERE id_docente = 2;
```

Innesca **ON DELETE!**

```
DELETE FROM Materia  
WHERE id_materia = 1;
```

Query Language

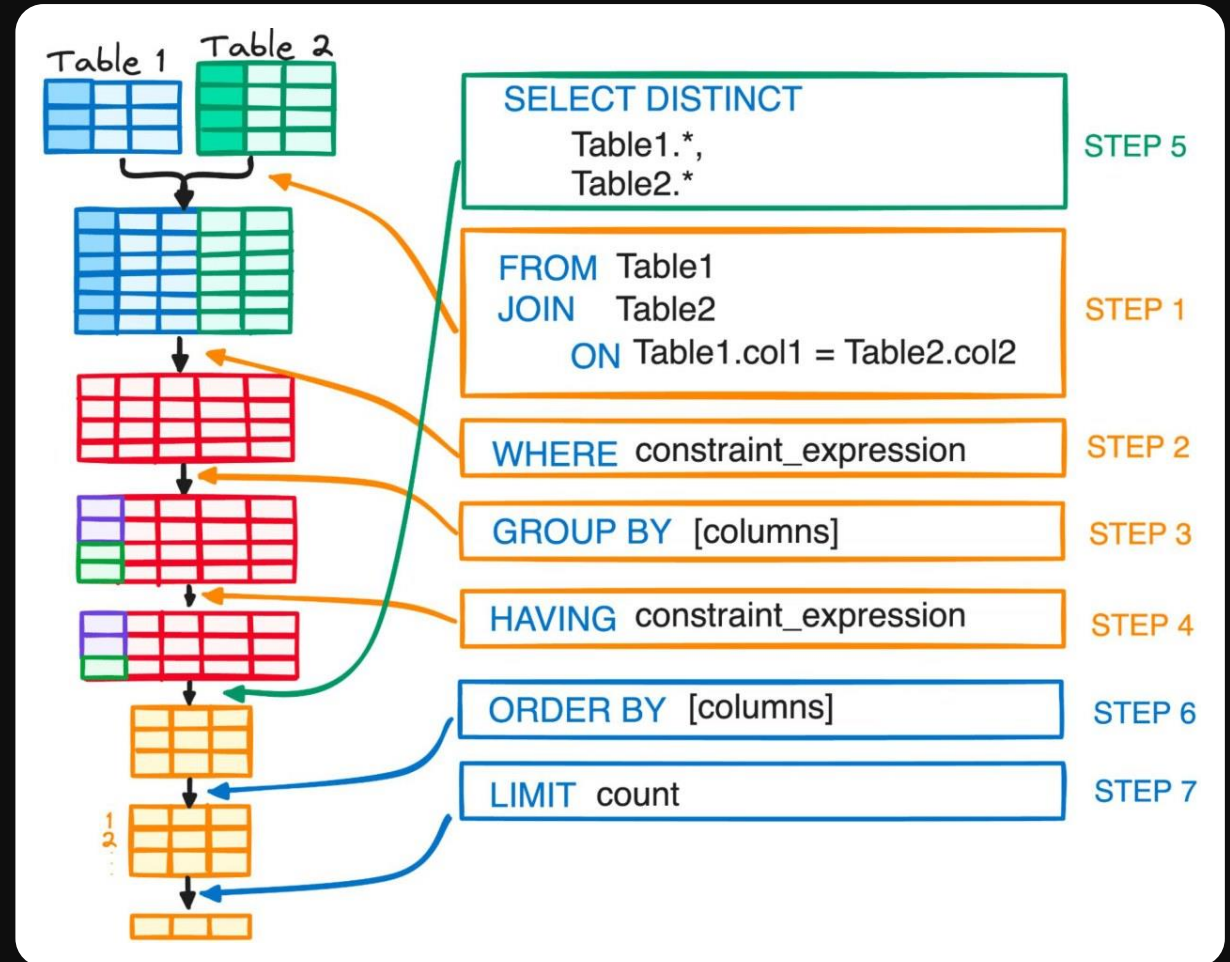
I comandi di base **ricalcano** gli operatori **dell'algebra relazionale**, che stavolta operano su **tabelle** invece che relazioni pure.

- **SELECT** [DISTINCT] [tab1.]<attr1> [**AS** <a1>][,[tab2.]<attr2> [**AS** <a2>]][, ...]
 - proiezione
 - rinominazione (AS)
- **WHERE** <condizioni_tuple>
 - selezione
- **FROM** <tab1> [T1] [,<tab2> [T2] [, <tab3> [T3] ...]]
 - tabelle sorgenti (con attribuzione di alias contestuale alla query)...
 - ... e prodotto cartesiano
- [FULL/INNER/RIGHT/LEFT] **JOIN** <tab2> [**ON** <tab1.id>=<tab2.id_tab1>] [**JOIN** ...]
 - giunzione/join su un attributo comune...
 - ... di solito la PK di una con la FK dell'altra

Query language e ordine di esecuzione

L'ordine con cui vengono **scritte** le varie parti (clausole) delle query QL **non coincide** con l'ordine di **esecuzione** nel DBMS!

- **differenza** con linguaggi **imperativi**, dove le istruzioni sono eseguite nell'ordine in cui compaiono.

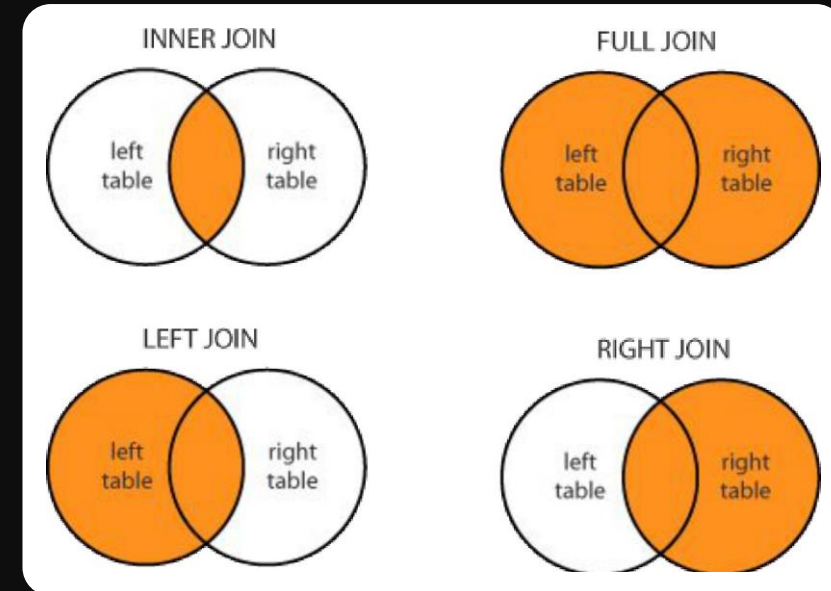


Tipologie di Join

Quando si uniscono due tabelle, **non sempre** c'è una **corrispondenza** tra i valori delle tuple che si cerca di unire.

La tabella risultante può cambiare in base alla tipologia di JOIN scelta.

- FULL/OUTER
 - su alcuni DBMS non c'è!
- RIGHT
- LEFT
- INNER
 - di solito è quello di default, se non si specifica la tipologia



Si possono combinare! Ad esempio:

- LEFT + INNER
- RIGHT + OUTER

Tipologie di Join

LEFT JOIN



Everything on the left
+
anything on the right that
matches

```
SELECT *
FROM TABLE_1
LEFT JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
```

RIGHT JOIN



Everything on the right
+
anything on the left that matches

```
SELECT *
FROM TABLE_1
RIGHT JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
```

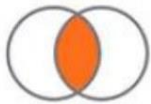
OUTER JOIN



Everything on the right
+
Everything on the left

```
SELECT *
FROM TABLE_1
OUTER JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
```

INNER JOIN



Only the things that match on the
left AND the right

```
SELECT *
FROM TABLE_1
INNER JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
```

ANTI LEFT JOIN



Everything on the left
that is NOT on the right

```
SELECT *
FROM TABLE_1
LEFT JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
WHERE TABLE_2.KEY IS NULL
```

ANTI RIGHT JOIN



Everything on the right
that is NOT on the left

```
SELECT *
FROM TABLE_1
RIGHT JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
WHERE TABLE_1.KEY IS NULL
```

ANTI OUTER JOIN



Everything on the left and right
that is unique to each side

```
SELECT *
FROM TABLE_1
OUTER JOIN TABLE_2
ON TABLE_1.KEY = TABLE_2.KEY
WHERE TABLE_1.KEY IS NULL
OR TABLE_2.KEY IS NULL
```